9

THE APPLICATION LAYER

The application layer holds the user (also known as application) programs that do the actual work for which the computers were purchased. These programs make use of the services offered by the presentation layer for their communication needs. However, certain applications, such as file transfer, are so common that standards have been developed for them to eliminate the need for each company to develop its own, and to make sure that everyone uses the same protocols. In this chapter we will examine several of these common applications in detail, both to see the general principles involved and to see how they fit in to the OSI Reference Model. We will also briefly look at some building blocks that the application layer provides for all applications to use.

9.1. APPLICATION LAYER DESIGN ISSUES

In this section we will provide a short introduction and overview of the applications to be studied in this chapter, just to get an idea of what kinds of issues must be dealt with in the application layer. These include file transfer, electronic mail, and virtual terminals, among others. Later in the chapter we will return to each topic to examine it in detail.

Furthermore, we will also briefly examine two building blocks that are often used by these and other applications. One of these deals with connection management and the other is concerned with concurrency.

9.1.1. File Transfer, Access, and Management

File transfer and remote file access are two of the most common applications in any computer network. People who are working together on a project commonly need to share files. One approach is to have a machine where the original of each file is held, and have copies transferred to other machines as needed. Another approach is to have each file "live" on the machine where it was created (or where it is maintained), and have users on other machines ask for copies when they need them.

Another situation in which file transfer is used is at a university that has many diskless workstations spread around campus along with one or more machines with large disks. Students can log into any workstation and access their files over the network. In an alternative design, the workstations can be equipped with small disks, allowing students to download their files to their workstations at the start of each terminal session and upload them back to the main machine when they are done. This arrangement means that a student can log into any workstation on campus (or perhaps even at the dorm), not just the one specific workstation where his files are.

Remote file access (as in the case of diskless workstations) is similar to file transfer, except that only pieces of files are read or written, rather than entire files. The techniques used for file transfer and remote file access are similar, so we will not make much of a distinction between them until we come to the subject of file replication. Furthermore, access to a file located on a remote computer that has its own users is hardly different from accessing a file on a dedicated **file server** machine that has no local users, so we will not make much of a distinction between these two cases either. For simplicity, we will assume that files are located on file server machines, with users on **client** machines wanting to transfer these files in whole or in part for reading and writing.

In contrast to the session and presentation layers, on which there has been very little work done outside of OSI, there has been a great deal of research about file transfer done at universities and in industry. Many file servers have been built and experimented with (Svobodova, 1984). As a result, our discussion of file transfer will reflect both OSI and nonOSI work.

The key idea behind most modern file servers is that of a **virtual filestore**, an abstract file server, either freestanding or running as a process on a timeshared computer. The virtual filestore presents a standardized interface to its clients, and provides a set of standardized operations that the clients can execute. Transfers to and from the virtual filestore use standardized protocols. If the real file server has a different internal structure than that of the virtual filestore, it will need some application layer software to hide the truth from the clients and make only the virtual filestore interface visible. By standardizing on a particular virtual filestore interface, as OSI has done, it is possible for application programs to access and transfer remote files without knowing all the details of numerous incompatible file servers.

In this chapter we will look at some of the characteristics of the OSI virtual filestore, as well as other ones, and also discuss some of the issues involved in designing and implementing file servers and file transfer protocols.

9.1.2. Electronic Mail

When the ARPANET first went into operation, its designers expected that process-to-process traffic would dominate. They were wrong. From its very inception, the volume of electronic mail between people has overshadowed the volume of communication between processes. While neither snow, nor rain, nor heat, nor gloom of night may stay the post office's couriers from the swift completion of their appointed rounds, the ARPANET's ability to deliver a message from coast to coast in a few seconds started a revolution in the way people communicate.

The attraction of electronic mail, of course, is that it is very fast. However, there are other advantages that are not as well known. The telephone also provides instantaneous access, but studies have shown that about 75% of all business calls fail to reach the intended party ("I am very sorry, but Mr. Smith is in a meeting/out of town/away from his desk."). Electronic mail has the speed of the telephone without requiring that both parties be available at the same instant. It also leaves a written copy of the message that can be filed away or forwarded. Furthermore, a message can be sent to many people at once.

Although electronic mail can be viewed as just a special case of file transfer, it has a number of specific characteristics not common to all file transfers. For one thing, the ultimate senders and receivers are nearly always people, not machines. This fact has resulted in electronic mail systems being constructed as two distinct, but closely related parts: one providing for the human interface (e.g., composition, editing, and reading mail), and one for transporting mail (e.g., managing mailing lists and providing notification of delivery).

Another difference between electronic mail and general purpose file transfer is that mail messages are highly structured documents. In many systems, each message has a large number of fields in addition to its content. These include the sender's name and address, the recipient's name and address, the date and time of posting, a list of people to receive carbon copies, the expiry date, importance level, security clearance, and many more.

Many telephone companies and PTT's are interested in offering electronic mail as a standard service to companies and individual subscribers. To prevent worldwide chaos, in 1984 CCITT defined a series of protocols for what it calls **MHS** (**Message Handling Systems**) in its **X.400** series of recommendations. ISO tried to incorporate them into the OSI application layer under the name **MOTIS** (**Message-Oriented Text Interchange Systems**), although such incorporation is not entirely straightforward given the lack of structure in X.400. In 1988, CCITT modified X.400 in an attempt to converge with MOTIS. We will discuss MOTIS

later in this chapter. Additional material on electronic mail can be found in (Huffman, 1987; Hutchison and Desmond, 1987; and Solman, 1987; Taylor, 1988).

9.1.3. Virtual Terminals

For some reason or other, terminal standardization has been a complete failure. Nearly all terminals accept certain character sequences, called **escape sequences**, for cursor motion, entering and leaving reverse video mode, inserting and deleting characters and lines, and so on. There are ANSI and other standards for these escape sequences, but nobody uses them. Each manufacturer has it own escape sequences, incompatible with those of every other manufacturer. Furthermore, the input (keyboard) problem is even worse than the output (screen) problem.

As a result, it is difficult for anyone to write a screen editor that works with an arbitrary keyboard and display. Even if a given terminal has a key labeled "insert character," it is very unlikely that an existing editor will be able to use that key for the function for which it was intended.

Similarly, if an airline reservations program displays a list of available flights, it would be nice if the user could just move the cursor to the desired flight using the arrow keys on the keyboard or the mouse and then hit the carriage return key or click a mouse button to select the flight. The airlines solve this problem by buying the main computer, all the terminals, and the software from a single manufacturer, so everything works together. However, as more and more people gain access to reservation systems from their terminals and personal computers over networks, the incompatibility problem can no longer be swept under the rug.

The OSI approach to solving it is to define a **virtual terminal**, which is really an abstract data structure that represents the abstract state of the real terminal. This data structure can be manipulated by both the keyboard and the computer, with the current state of the data structure being reflected on the display. The computer can query the abstract data structure to find out about keyboard input and can change the abstract data structure to cause output to appear on the screen. In the section on virtual terminals we will describe in some detail how this idea works.

9.1.4. Other Applications

Numerous other applications have been or are being standardized. Many of these pertain to some specific industry, such as banking. Still, there are a few others that are general enough to warrant saying at least a few words about in this book. Among these are directory services, remote job entry, graphics, and telematics.

Directory services are the electronic equivalent of the telephone book. They provide a way to find the network address of people and services available on the network. Even leaving aside the question of whether that address should be the

NSAP, TSAP, SSAP, PSAP, or something else, there are a variety of interesting design issues for directory service. We will discuss these in this chapter.

Remote job entry allows a user working on one computer to submit a job for execution on another computer. Typically it is the user of a personal computer submitting a batch job for execution on a large mainframe somewhere else. In many cases, the program, data files, and job control statements must all be collected, possibly from diverse machines, bundled together and submitted as a unit. Finally, the output must be directed to the appropriate destinations.

Not all applications are text oriented. Some primarily use drawings. Thus there is a need to send, for example, engineering drawings over networks for remote display and plotting. Considerable work has been done in this area and we will discuss some of the ideas behind it later.

Telematics is the collective name for public information services for home and office use. **Teletex** is a simple system in which a small amount of information can be sent to a large number of people using conventional television broadcasting. **Videotex** (formerly called **viewdata**) is an interactive service in which users can access large public data bases and perform simple transactions like making reservations.

9.1.5. OSI Service Elements—ACSE and CCR

After ISO had been working on the above and other applications for a while, it became apparent that many applications had a certain amount in common. Almost all of them needed to manage connections and many had to coordinate activities among three or more parties. To avoid forcing each new application to solve these programs all over again, ISO decided to standardize some of these building blocks. Below we will describe two of the most important ones.

Association Control Service Element

ACSE (Association Control Service Element) is designed to manage connections, which in the application layer are called **associations**. Figure 9-1 shows the ACSE primitives.

Each ACSE primitive maps one to one with the corresponding presentation layer service primitive. Consequently, one could legitimately ask what use they are. Surely applications could just as well use the presentation primitives directly. Two reasons are offered for their existence. The first one is symmetry. All the other layers have their own primitives for connection establishment, so it would be strange if the application layer did not. The second reason is that future changes to ACSE may give *A-ASSOCIATE* more work to do, for example, user authentication. Such a function is clearly inappropriate for the presentation layer.

OSI ACSE Primitive	Description
A-ASSOCIATE	Establish an association
A-RELEASE	Release an association
A-ABORT	User-initiated abort
A-P-ABORT	Provider initiated abort

Fig. 9-1. The ACSE primitives.

Commitment, Concurrency, and Recovery

CCR (Commitment, Concurrency, and Recovery) is a service element that coordinates multiparty interactions in a foolproof way, even in the face of repeated system crashes. Nearly all applications that need to operate reliably use CCR.

To see the kind of problem that CCR has been designed to solve, consider a simple banking transaction. Marvin has instructed his bank to transfer 1 million dollars from his account in Amsterdam to the account of Simon at a different bank in distant Amstelveen. The instruction is passed to the electronic clearinghouse that handles all interbank transfers.

The straightforward protocol is for the clearinghouse to simultaneously send messages to both banks instructing them to debit or credit the appropriate account and then acknowledge that they have done so. The trouble comes in when one of the banks does it and the other is unable to for some reason such as a network failure. What we need is a protocol that either completely succeeds or does nothing at all, rather than one that does part of the work and leaves the banking system in an inconsistent state.

CCR solves this problem by providing atomic actions. An **atomic action** is a collection of messages and operations that either completely succeeds or can be rolled back to the original state, as if no actions at all had occurred.

Atomic actions are implemented using **two-phase commit**, a technique first used in distributed data bases, but which is applicable to almost any multiparty operation. It is illustrated in Fig. 9-2.

In the first phase, the master (the clearinghouse in our example) tells each slave (other bank) what it wants done. Each slave checks to see if it can carry out its request. If it can, it records the request on **stable storage** (some medium, such as a replicated disk, that is known to survive crashes), locks the data so that no other requests from other masters can interfere, and reports back success. It also records the initial state of the data (e.g., the balance in the account) on stable storage. On the other hand, if a slave cannot execute the request, it just reports back failure immediately and does nothing else.

When all the replies have come back, the master sees if all the slaves can perform their assigned tasks. If so, it sends each one a **commit** message, instructing it to go ahead and do the work. Since message loss is handled by lower layers, the

```
Master                                          Slave

Begin atomic action;
Send Request 1;
   :
Send Request n;
Send "Prepare to commit" message;
                                          if action can be performed
                                           then
                                            begin
                                              Lock data;
                                              Store initial state on disk;
                                              Store requests on disk;
                                              Send "OK" message
                                            end
                                           else
                                              Send "Failure"
if all slaves said "OK"
   then send "Commit" message
   else send "Rollback" message;
Wait for acknowledgements
                                          if master said commit
                                           then begin
                                              Do work;
                                              Unlock data;
                                            end;
                                          Send "Acknowledgement" message
```

Fig. 9-2. Two phase commit.

only thing that can go wrong is that a slave can crash before carrying out its work. However, it can recover when it comes back up by looking at stable storage to see what the initial state was and what it was supposed to do.

If any of the slaves report failure back in the first phase, the master aborts the action and tells all the slaves to unlock their data and restore the initial state. The net result is that no work is done and no data are changed.

CCR provides primitives for each of the essential actions in the two-phase commit. They are shown in Fig. 9-3.

OSI CCR primitive	From	Description
C-BEGIN	Master	Begin an atomic action
C-PREPARE	Master	End of phase 1; prepare to commit
C-READY	Slave	Slave is able to do its work
C-REFUSE	Slave	Slave is not able to do its work
C-COMMIT	Master	Commit the action
C-ROLLBACK	Master	Abort the action
C-RESTART	Either	Announce that a crash occurred.

Fig. 9-3. The CCR primitives.

The first six have already been discussed. The last one is used when either the master or a slave has rebooted after a crash. Its purpose is to reset the state of the action back to its initial state so it can be repeated.

9.2. FILE TRANSFER, ACCESS, AND MANAGEMENT

File handling is one of the principal services in any network or distributed system. In this section we will look at file server design, concurrency control, file replication, and some key implementation issues. We will take examples from various sources (including the OSI virtual filestore), but we will look specifically at OSI later in this chapter. Considerable literature on file servers exists. A few papers are: (Brown et al., 1985; Hagmann, 1987; Howard, 1988; Rodriguez et al., 1986; Sidebotham, 1986; Svobodova, 1984; Vandome, 1986; Zwaenepoel, 1985).

9.2.1. File Servers

A file server (or a virtual filestore) can be characterized by three properties:

1. File structure.
2. File attributes.
3. File operations.

We will now describe each of these in turn.

Every file server has some conceptual model of what a file is. Different file servers have different models. Three models, however, are widely used. In the first model, a file is an unstructured lump of data without any substructure known to the file server. Since the file server does not know anything about the internal structure of its files, it cannot perform any operations on parts of files. Typically the only operations possible in this model are reading or writing entire files. Fig. 9-4(a) illustrates an unstructured file.

The next step up is the **flat file**, which consists of an ordered sequence of records. The records of a file need not all be of the same size and type. Furthermore, some or all of the records may have labels (keys) associated with them. These labels are not part of the data of the file, just as the file name is not either. With these files, it is possible for clients to address specific records, either by their labels or their positions, so the file server may support operations on individual records such as extending, replacing, or deleting them. A flat file is shown in Fig. 9-4(b). A UNIX file, for example, can be regarded as being a sequence of 1-byte records that are individually addressable by position.

The most general model of a file is the **hierarchical file**, which takes the shape of a tree. Each node of the tree may have a label, a data record, both, or neither.

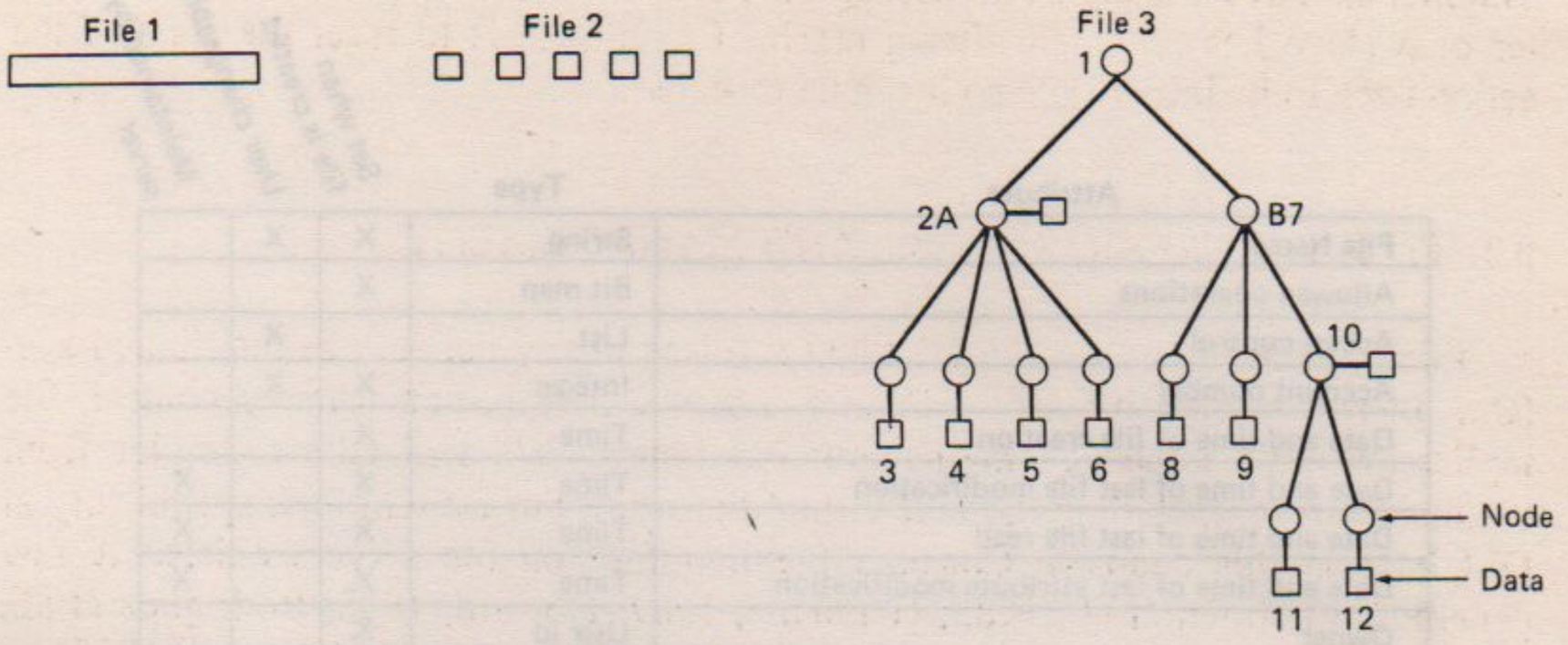


Fig. 9-4. Files. (a) Unstructured. (b) Sequence of records. (c) Hierarchical.

Fig. 9-4(c) depicts a hierarchical file with 12 nodes, two of which have labels (*A*, and *B*), and 10 of which have data records (all but nodes 1 and 7).

It is frequently useful to have a standard ordering for all the nodes, not only to allow unlabeled nodes to be addressed by node position for delete and replace operations, but also for transferring entire files. In this example, the nodes are numbered by depth-first search, in which the tree is walked by always visiting the leftmost unvisited node next. Alternatively, if all the nodes are labeled, a specific node can be addressed by giving its path from the root (provided that all sister nodes have unique labels). It is also worth pointing out that node indices or labels also serve for addressing entire subtrees. Thus an operation to delete a subtree applied to node 10 would also remove nodes 11 and 12.

All files have **attributes** that describe them. As a bare minimum, each file must have a name or other identifier, and a size telling how much storage it currently occupies. In practice, most file servers maintain various other attributes as well. Figure 9-5 shows the OSI virtual filestore attributes as an example. Each attribute has a name, a type, and a value (possibly a set or list).

Some attributes are created when the file is created, and are forever frozen thereafter. Others can be explicitly changed by user operations. Still others (e.g., time of last modification) are automatically maintained by the file server. Which attributes are user changeable varies from file server to file server.

Although most of the attributes are straightforward, a few of them require some comment. The *Allowed operations* attribute allows the creator of a file to specify, for example, that some operations are not valid on this file. Insertion of records in the middle of UNIX files could be prohibited in this way.

Access control determines who may access the file, and how. We will discuss this important issue in detail later.

The *File available* attribute can be used to indicate that there may be a slight delay when opening the file (e.g., to fetch it from a bank vault on the other side of

Attribute	Type	Set when file is created	User changeable	Maintained by server
File Name	String	X	X	
Allowed operations	Bit map	X		
Access control	List		X	
Account number	Integer	X	X	
Date and time of file creation	Time	X		
Date and time of last file modification	Time	X		X
Date and time of last file read	Time	X		X
Date and time of last attribute modification	Time	X		X
Owner	User id	X		
Identity of last modifier	User id	X		X
Identity of last reader	User id	X		X
Identity of last attribute modifier	User id	X		X
File availability	Boolean	X	X	
Contents type	Object id	X	X	
Encryption key	String	X		
Size	Integer	X		X
Maximum future size	Integer	X	X	
Legal qualifications	String	X	X	
Private use	String	X	X	

Fig. 9-5. The OSI virtual filestore attributes.

town). The *Contents type* attribute could tell something about the record structure, for example, by giving an object identifier corresponding to the ASN.1 definition of its records.

The *Legal qualifications* attribute is a string that might contain information relating to various national privacy laws, for example: "This file contains personal information about Swedish citizens and may not be transferred outside of Sweden." Finally, *Private use* is a catchall for any information not provided for by other attributes (e.g., whether or not the information in the file is trustworthy enough to be worth believing).

File operations can apply to a file as a whole or to its contents, that is, to the individual records. Figure 9-6 gives a list of possible operations (also taken from the OSI virtual filestore). The column labeled *Bitmap* shows which operations are included in the *Allowed operations* attribute of Fig. 9-5. For example, a file can be created with permission to read it, but not permission to insert, replace, extend, or erase any of its records. Thus the file cannot be changed after it has been created.

In addition to the operations listed in Fig. 9-6, some file servers also support directory operations. Clients can create and delete directories, and move files about from one directory to another.

Operation	Applies to whole file	Applies to contents	Bitmap	Description
Create file	X			Create a new file
Delete file	X		X	Destroy an existing file
Select file	X			Pick a file for attribute management
Deselect file	X			Terminate the current selection
Open file	X			Open a file for reading or modification
Close file	X			Close an open file
Read attribute	X		X	Read a file attribute
Change attribute	X		X	Modify a file attribute
Locate		X		Locate a record
Read		X	X	Read data from the file
Insert		X	X	Insert new data in the file
Replace		X	X	Overwrite existing data
Extend		X	X	Append data to some record
Erase		X	X	Delete a record

Fig. 9-6. The OSI virtual filestore operations.

All file servers must deal with access control and protection in some way. The simplest and least reliable way is to assume that all the client machines are trustworthy and just carry out all requests that come in. If all the client machines are large mainframe computers with elaborate protection schemes of their own that prevent users from even making unauthorized server requests, this approach may be justified. In a wide-area X.25 public network, the caller's address provided by the carrier during connection setup may be sufficient to authenticate the caller as a trusted mainframe. On the other hand, trusting personal computers on a LAN is foolhardy, so a better method is needed for that case.

One such method is to verify the sender of each request, either by having the sender include a password in each request, or by using one of the digital signature methods discussed in Chapter 8. If wiretappers are potentially a problem, a cryptographically-based digital signature is much better than just sending a plaintext password, unless, of course, the entire session is encrypted. Once the server knows the identity of the client, it can use any of the traditional schemes used by time-sharing systems for protection. For example, the owner could designate the access rights for himself, other members of his group, and everyone else, as in UNIX, or for each file he could give a list of who can access it.

A more elaborate method is to have one or more passwords per file (e.g., one for reading and another for writing). In this system, anyone presenting a valid password is allowed to perform the corresponding operations, without regard to the sender's true identity. Sparse capability-based protection, with a bit map to indicated permitted operations (Tanenbaum et al., 1986), is a variant of this idea.

9.2.2. Concurrency Control

Network file servers have multiple clients to take care of. If two or more clients should accidentally happen to decide to access the same file at more or less the same time, problems can occur. Consider the following scenario. Two clients open the same file for writing. Then each of them simultaneously issues a request to replace the first record of the file. Whichever request happens to get serviced last wins, and overwrites the other one. It is difficult to build reliable applications under these conditions.

One widely implemented solution is to permit clients to lock files before using them. Two kinds of locks are used, **shared locks** and **exclusive locks**. When a client requests a shared lock on a file, typically at the time the file is opened, the lock will be granted only if the file is unlocked or has other shared locks on it. If there is an exclusive lock on it, the open request cannot be granted. Exclusive locks are only granted on unlocked files.

Shared locks are typically used for reading; exclusive locks are normally used for writing. When reading a file, a client generally does not mind the existence of other readers, but wants the shared lock simply to prevent the file from being changed while it is busy. However, when writing a file, the client needs to make sure no other readers or writers are active.

When a file is in heavy use by multiple clients, mostly for reading but occasionally for writing, a problem can occur if ungrantable requests are just rejected. As time goes on, some readers finish, but new readers enter, so that there are always readers active and no writers can ever get started. As an alternative to rejecting requests for exclusive locks out of hand, the server could queue them, and also queue any new requests for shared locks. When the current readers finally finish, the first queued writer is then given a chance. After the writer finishes, the server would have to make a policy decision about whether writers or readers should be allowed to go next.

The granularity of the locking is an important design issue. Obviously it is possible to lock entire files, but it is also possible to lock specific records or subtrees. The finer the grain of the locking, the more concurrency is possible. If two clients need to update the same file, there is no reason to stop them from doing it simultaneously if they are working on disjoint subtrees that can be independently locked.

Locking introduces several annoying problems. First, if a client needs to access several files, as is commonly the case when transferring money in banking applications, there is a potential for deadlock. If client 1 has a lock on file *A* and client 2 has a lock on file *B* and each one is trying to get the other file, neither will ever succeed. Ad hoc solutions are sometimes possible, such as having all clients lock files in alphabetical order, thus preventing cycles, but in general deadlock avoidance is up to the clients, not the file server.

Another problem with locking is what happens if a client holding some locks crashes? Unless something is done, the locked files will remain locked forever. If

the server is not informed about client crashes, the only thing it can do is adopt a policy of automatically breaking locks on files that are not accessed for some specified time interval. However, if a client is too slow, it may discover that some of its locks have timed out part way through a complex multifile update, leading to chaos.

As an alternative to having clients set individual locks, some file servers support atomic actions, often called **transactions** in the context of file servers. When this facility is available, a client can tell the server to begin a transaction, followed by any number of opens and file operations, and finished by a command to end the transaction. It is up to the server to carry out all the requests in an atomic (i.e., indivisible) way, without interference from other client requests. If the client decides to abort the transaction, or if it times out or something else goes wrong, all files are restored to the state they had before the transaction started.

Transactions can be implemented by having the server make a copy of each file that is opened for writing. Changes that are made by subsequent writes are made on the copy, not the original. If the transaction completes successfully, the copies replace the originals. If multiple servers are involved, the CCR techniques described earlier in this chapter can be used.

If two clients use the same file in simultaneous transactions, the first one to finish wins and makes all its changes, and the second one is aborted. Although this strategy means that the second client has wasted some CPU time, he is effectively rolled back to the start of his transaction, so no damage has been done to any of his files. It is as though he never even started. The beauty of the transaction model is that there are no user-visible locks and the server can prevent deadlock.

The entire discussion of concurrency control so far has been centered around the problem of what to do when multiple users want to update the same file. A completely different approach is to prevent all updates. In this view, files are **immutable**. Once a file has been created, it can never be changed.

To permit new information to be incorporated into a file, a client can create a new **version** of a file, which replaces (but does not modify) the original. Thus a file becomes a sequence of immutable versions. When a client opens a file, the most recent version is opened. Subsequent reads use that version for consistency, even if a new version has been created by another client in the meanwhile. Since creating a new version is an atomic operation, there are no problems with files containing updates from multiple clients. Mullender and Tanenbaum (1985) describe the FUSS system, a multiversion file server, in some detail.

9.2.3. Replicated Files

Having tackled file systems with a single server and multiple clients, let us now look at systems with multiple servers and multiple clients. Networks often have multiple file servers for several reasons:

1. To split the workload over multiple servers.
2. To allow file access to occur even if one file server is down.
3. To increase reliability by having independent backups of each file.

One strategy for file replication is to let each user open accounts with as many file servers as he wants, and manage all the replication himself. This puts a considerable administrative burden on the user, and human nature being what it is, not much replication is to be expected. Nevertheless, when a server crash destroys a users' files, statements from the computer center management to the effect, "It's your own fault," are not likely to be well received.

Thus we would prefer the replication to be done automatically by the file servers themselves. As long as files are never modified, maintaining multiple copies is easy. The trouble comes when one copy is updated: the others must be updated too. The simplest solution, and one which is widely used in practice, is **primary copy replication**. In this scheme, one copy is designated as the master, and all the others are slaves. Updates are always made to the master, which then propagates them to the slaves. Although this method is simple and unambiguous, it has the serious disadvantage of making updates impossible when the master is currently unavailable, either due to a crash of its server or a partition of the network (e.g., due to a broken gateway).

A more robust method, especially with multiple clients active, is **voting** (Gifford, 1979). Call the number of servers with copies of a file N. To read a file, it is required to obtain a **read quorum**, N_r. To modify a file, one needs a **write quorum**, N_w, subject to the constraint $N_r + N_w > N$. Only after the appropriate number of servers have been asked if they are willing to participate and agree, can the operation be performed.

The point of requiring quora can be seen by an example. In Fig. 9-7(a) we have $N = 9$, $N_r = 2$, and $N_w = 8$. Suppose one client has acquired two servers, A and B for reading. In order for another client to perform an update, a write quorum of eight servers must be acquired, but this is impossible since there are only nine of them and two are already busy. Would-be writers must wait until the reader has finished. (As an aside, the reader need not read one copy from each server; a single copy is enough.)

Similarly, in Fig. 9-7(b) we have a client who has acquired five servers for writing, precluding any reads, which also need five. In general, making N_r low means that reads are easy, but writes are hard, and vice versa. The optimal choice depends on whether reads or writes are more common.

Since writes do not update every copy of the file, a read quorum will usually contain some obsolete copies as well as the latest one. In order to allow readers to tell which is the most recent one, each copy maintains a version number. Each write creates a new version with a number higher than any previous version, so that

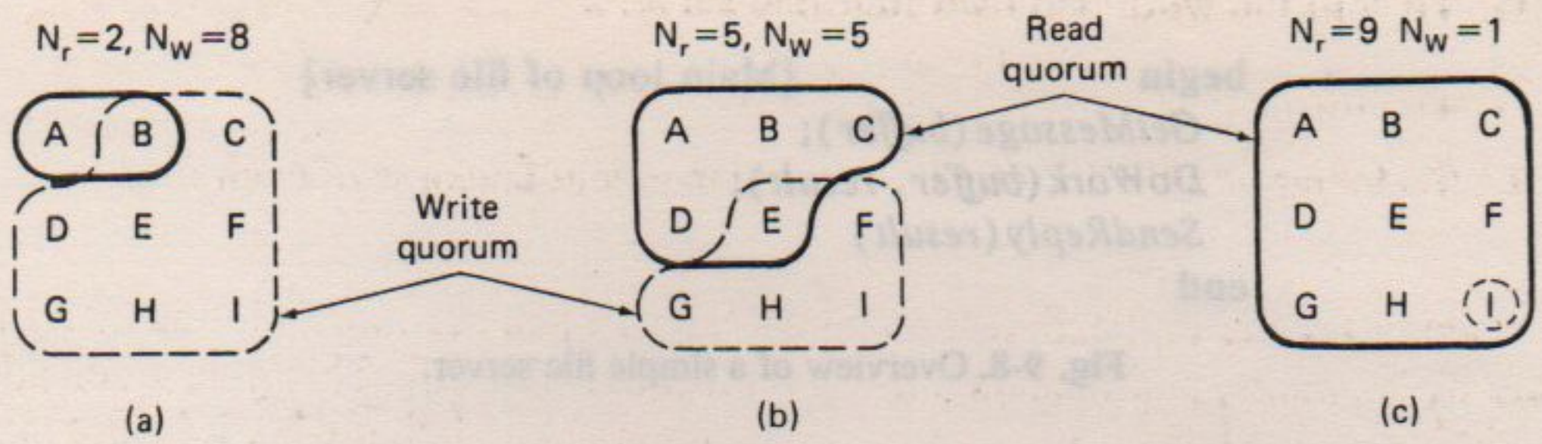


Fig. 9-7. Three examples of a read quorum and a write quorum.

when a reader gets a quorum it can tell which copy or copies are up-to-date (there is always at least one).

An interesting variation on voting is **voting with ghosts** (Van Renesse and Tanenbaum, 1988). In most applications, reads are much more common than writes, so N_r is typically a small number and N_w is nearly N. This choice means that if a few servers are down, it may be impossible to obtain a write quorum at all.

Voting with ghosts solves this problem by creating a dummy server, with no storage, for each real server that is down. A ghost is not permitted in a read quorum (it does not have any files, after all), but it may join a write quorum, in which case it just throws away the file written to it. A write only succeeds if at least one server is real.

When a failed server is rebooted, it must obtain a read quorum to locate the most recent version, which it then copies to itself before starting normal operation. The algorithm works because it has the same basic property as the basic voting scheme, namely, N_r and N_w are chosen so that acquiring a read quorum and a write quorum at the same time is impossible. The only difference here is that dead machines are allowed in a write quorum, subject to the condition that when they come back up they immediately obtain the current version before going into service.

Other replication algorithms are described in (Bernstein and Goodman, 1984; Brereton, 1986; Pu et al., 1986; and Purdin et al., 1987).

9.2.4. Implementation Issues

File servers are so important and widespread that it is worth saying a little bit about how they are implemented. The simplest way is to organize the server as a single process, with the main loop as shown in Fig. 9-8. Requests for work come in, are processed, and the results are sent back. The server handles one request at a time. While it is busy with the current request, all new requests are simply queued until the server is ready.

The approach of Fig. 9-8 is certainly usable, but it has the disadvantage that if the processing of a request requires several disk accesses (for directories and

```
begin                          {Main loop of file server}
  GetMessage(buffer);
  DoWork(buffer, result);
  SendReply(result)
end
```

Fig. 9-8. Overview of a simple file server.

various internal tables) the server will be idle while waiting for the disk accesses to complete. A more sophisticated design that has higher performance is illustrated in Fig. 9-9. In this design the file server is split up into several **tasks** that share a common address space. Each task is a separate thread of control, with its own program counter and its own stack. However, the tasks share global data, such as file system tables and buffers.

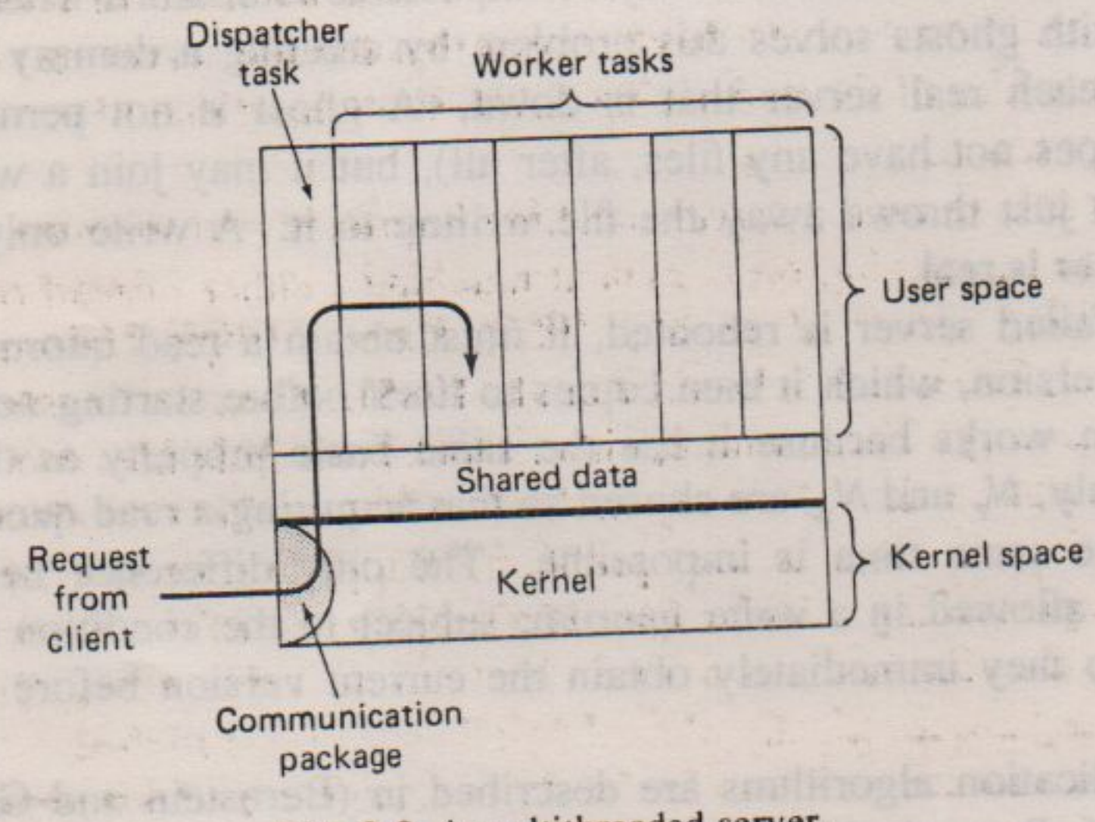


Fig. 9-9. A multithreaded server.

When a request message from a client comes in, the kernel accepts it and passes it to the **dispatcher task**, which is normally waiting. The dispatcher then inspects it and hands it off to an idle **worker task**. Since the dispatcher and worker tasks are in the same address space, only a pointer has to be passed, not the message.

Each task handles a single request to completion. However, if a task blocks while waiting for the disk, another task can run. In this way, the CPU is never idle when there is work to do. When a worker finishes its assigned job, it updates its entry in the master status table in global memory, so the dispatcher will know that it is available to handle another request.

Most file servers are block oriented, that is, they divide the disk up into blocks (e.g., 1K bytes) and represent files as sequences of blocks. A bit map or free list keeps track of blocks unassigned to any file. When a file grows, a free block is

allocated and assigned to the file. Some file servers attempt to keep all the blocks of a file within a few cylinders to reduce arm motion, but in general the blocks are not consecutive on the disk.

A widely used technique to improve performance is to have the server maintain a **buffer cache** in memory (e.g., as part of the shared data in Fig. 9-9). The buffer cache consists of the n most recently used disk blocks. When a disk block is read in, it is put in the cache, and the least recently used block is purged (after being rewritten to the disk if it has been modified). If the cache is large enough, the hit rate can be 90 percent or more, thus eliminating a large fraction of the disk accesses.

A completely different approach is taken in the Amoeba system. The Amoeba file server is not block oriented. Instead, it stores each file contiguously on the disk. The cache is also organized this way, with entire files cached, each file occupying some number of consecutive bytes in the server's memory. Files can be transferred between disk and memory in a single disk operation. When a client requests a file that is in the cache, the entire file can be sent as a sequence of back-to-back packets. Measurements on Amoeba have shown that the file server can operate at close to the full bandwidth of the underlying LAN, something that block-oriented servers cannot even approach.

Although caching on the server machine improves performance, an even bigger gain can be had by caching on the client machines, either instead of or in addition to server caching (Howard et al., 1988; Kazar, 1988; Nelson et al., 1988; Ousterhout et al, 1988; and Schroeder et al., 1985). The client caches can be either on disk or in memory. Furthermore, either blocks or whole files can be cached.

When files can be shared among multiple users working on different machines, client caching introduces a serious problem. Suppose several client machines all have cached copies of a particular file. If a user on one of these machines modifies the local copy, the changes will not be visible to the other users. Thus the semantics of file sharing are drastically altered by allowing local caches.

One approach to this problem is simply to accept it, and tell users to be careful. Some commercial systems actually work this way, but it is not an esthetically pleasing solution.

A second attempt is to have the file server keep track of which client has which blocks and files in its cache. Whenever a client modifies a block or file, it tells the file server, so the latter can send an **unsolicited message** to every client holding the modified block or file, instructing it to remove the now-invalid block or file from its cache.

A third possibility is to cache entire files only, and to have the server keep track of which files are cached where and whether they are opened for reading or for writing. If a file has one writer and no readers, everything works fine. If a second user opens that file, the file server sends an unsolicited message to the client machine asking it to upload the modified file back to the server and to cease caching it. The second open is suspended until the upload has completed.

In effect, the file server maintains a central data base of file usage and simply tells clients to disable caching of all shared files that are being modified. Caching is only permitted for read-only files or files with a single writer and no readers. This scheme is only marginally more elegant than the other ones, but at least it is reasonably efficient because in practice files that are heavily shared are rarely updated.

We have saved the best part for last. In the networking world, there is a tremendous controversy about the issue of connectionless vs. connection-oriented file servers. In the connectionless model, one has a **stateless file server**. To read or write (part of) a file, a client sends a request specifying the file, the record position or label, and the amount of data to be transferred. Files do not have to be opened before use or closed after use. Each request is fully self contained. The SUN NFS system is a widely-used stateless server.

In contrast, a connection-oriented server maintains internal state. When a file is opened, the server makes a table entry for the newly-opened file. Usually the index of this entry is returned to the client for use in subsequent requests.

In practice, files are normally read sequentially, so somebody has to keep track of the current file position. In the stateless model, it is the client who maintains the file position and sends it along on each request. In the other model (sometimes called **stateful** although the word seems fairly contrived), it is the server that keeps track of the file position, so the client can just give the table index, to identify the file, and the byte count.

Each model has advantages and disadvantages. The stateless server's main attraction is that it is highly robust. If it crashes and then reboots, there is no state information to lose, so the only thing a client might notice is a longer response time while the server is coming back up. In contrast, a crash of a stateful server effectively closes all open connections and puts full responsibility for recovery on the clients.

The counterargument against the stateless server is that it is difficult to implement normal file system semantics in a stateless way. For one thing, file locking is inherently stateful. If a server crashes and forgets which files are locked, that can hardly be made transparent to the clients.

For a second thing, suppose a client opens a file and then removes it while the file is open (something UNIX programs frequently do with scratch files). With a stateful server, the file can still be used by referring to the table index, but other programs cannot get at the file, which of course, was the purpose of the whole maneuver.

As a third example, suppose a client opens a file and then changes the protection mode of the file so that it is no longer permitted to access the file. Again, as in the second example, with a stateful server there is no problem in continuing to access the already open file, but with a stateful server it is difficult, if not impossible, to achieve the original semantics.

The adherents of the stateless model would argue that these traditional semantics are actually rather bizarre, but due to the way they were implemented on

centralized systems, they seemed natural and no one ever complained. Now that their limitations have become painfully obvious, the solution is to change the semantics of file access to something more reasonable.

The stateless versus stateful debate will no doubt go on for years. Taking sides will be left as an exercise for the reader. It is worth making one final point however. Be sure you realize that there is a subtle difference between the concepts of "connectionless" and "stateless." Connectionless refers to communication, whereas stateless refers to file system design. Theoretically one could imagine a stateless, connection-oriented server that accepted fully-specified requests over its connection, but did not maintain any state between requests. Such a design makes little sense, but it is possible, for example, to access an NFS server over an X.25 network. The reverse situation, sending unreliable datagrams to a stateful server, boggles the mind.

9.3. ELECTRONIC MAIL

Electronic mail, or **e-mail**, as it is known to its many fans, has been around for over two decades. The first e-mail systems simply consisted of file transfer protocols, with the convention that the first line of each message (i.e., file) contained the recipient's address. As time went on, the limitations of this approach became more obvious. Some of the complaints were:

1. Sending a message to a group of people was inconvenient.
2. Messages had no internal structure, making computer processing difficult. For example, if a forwarded message was included in the body of another message, extracting the forwarded part was difficult.
3. The originator (sender) never knew if a message arrived or not.
4. If someone was planning to be away on business for several weeks, and wanted all incoming mail to be handled by his secretary, this was not easy to arrange.
5. The user interface was poorly integrated with the transmission system requiring users first to edit a file, then leave the editor and invoke the file transfer program.
6. It was not possible to create and send messages containing a mixture of text, drawings, facsimile, and voice.

As experience was gained, new proposals were made for more ambitious e-mail systems (Bauerfeld et al., 1984; Horak, 1984; Rose et al., 1985). In 1984, CCITT

drafted its X.400 recommendation, which was later taken over as the basis for OSI's MOTIS. In 1988, CCITT modified X.400 to align it with MOTIS. Unlike, say, FTAM, which is widely perceived as being unwieldy and which will probably never replace "native" file systems, MOTIS/X.400 is rapidly becoming the dominant form for all electronic mail systems. For that reason, we will concentrate our discussion of electronic mail on it.

9.3.1. Architecture and Services of MOTIS and X.400

In this section we will provide an overview of what e-mail systems, especially MOTIS and X.400, can do and how they are organized. For the sake of brevity, we will refer to them by their OSI name, MOTIS (ISO 10021), but everything in this section also holds for X.400 as well. MOTIS is concerned with all aspects of the electronic mail system, starting at the time the originator decides to write a message, and ending with the time the recipient (receiver) finally throws it in the garbage can. Let us now briefly describe six of the basic aspects of any electronic mail system.

Composition refers to the process of creating messages and answers. Although any text editor can be used for the body of the message, the system itself can provide assistance with addressing and the numerous header fields attached to each message. For example, when answering a message, the mail system can extract the originator's address from the incoming mail and automatically insert it into the proper place in the reply.

Transfer refers to moving messages from the originator to the recipient. In large part, this requires interfacing with ACSE or the presentation layer to establish the necessary connection, output the message, and release the connection. The mail system should do this completely automatically, without bothering the user.

Reporting has to do with telling the originator what happened to the message. Was it delivered? Was it rejected? Was it lost? Numerous applications exist in which confirmation of delivery is important and may even have legal significance, ("Well, Your Honor, my mail system is not very reliable, so I guess the electronic subpoena just got lost somewhere").

Conversion may be necessary to make the message suitable for display on the recipient's terminal or printer. This feature is especially important because the world is full of existing, incompatible sending and receiving devices. For example, sending a message from a computer terminal to a facsimile terminal might require projecting the message onto a screen, then scanning the screen pixel by pixel to build up a facsimile image for the recipient's terminal.

Formatting pertains to the form of the displayed message on the recipient's terminal. This book was typeset using *troff*. If the input file were to be transmitted by e-mail, it would have to be reformatted at the receiving end to give it the desired appearance. Although the message could be saved to a file, *troffed*, and then examined with a special previewer program, it is a lot simpler, especially for unsophis-

ticated users, if the mail system can just display the formatted message directly (very possibly by calling the formatting program itself).

Disposition is the final step and concerns what the recipient does with the message after receiving it. Possibilities include throwing it away immediately, reading it first then throwing it away, reading it then saving it, and so on. There should also be a way to retrieve saved messages to reread them, forward them, or process them in other ways.

In addition to these basic services, most mail systems provide a large variety of advanced features. Let us just briefly mention a few of these. When people move, or when they are away for some period of time, they may want their mail forwarded, so the system should be able to do this automatically.

On the other hand, if someone has decided to take a vacation in Tahiti to "get away from it all" for a few weeks, he may *not* want his mail forwarded. Instead, he may want the mail system to send a canned reply to the originator of each incoming message saying that he is away and telling when he will return. However, the mail system should keep track of who has been sent a reply and not send a second reply even if a second message comes in from the same originator.

Most mail systems allow users to create **mailboxes** to store incoming mail. Commands are needed to create and destroy mailboxes, inspect the contents of mailboxes, insert and delete messages from mailboxes, and so on.

Corporate managers often need to send a message to each of their subordinates, customers, or suppliers. This gives rise to the idea of a **distribution list**, which is a list of electronic mail addresses. When a message is sent to the distribution list, identical copies are delivered to everyone on the list.

Registered mail is another important idea, to allow the originator to know that his message has arrived. Alternatively, automatic notification of undeliverable mail may be desired. In any case, the originator should have some control over the reporting of what happened.

Other advanced features are carbon copies, high-priority mail, secret (encrypted) mail, alternative recipients if the primary one is not available, and the ability for secretaries to handle their bosses' mail.

A key idea in all modern e-mail systems is the distinction between the **envelope** and its contents. The envelope encapsulates the message, just as a TPDU encapsulates an SPDU. The envelope contains parameters needed for transporting and interpreting the message, such as the destination address, priority, security level, and terminal type required, all of which are distinct from the message itself. Envelopes and messages are illustrated in Fig. 9-10.

Mail systems generally distinguish three types of messages: user messages, replies, and probes. **User messages** contain information sent from one user to another. These are the most important messages carried by the mail system, and are its reason for existence. They can be arbitrarily long and can contain anything the originator chooses to include.

Replies are system-generated messages sent back to an originator to report on

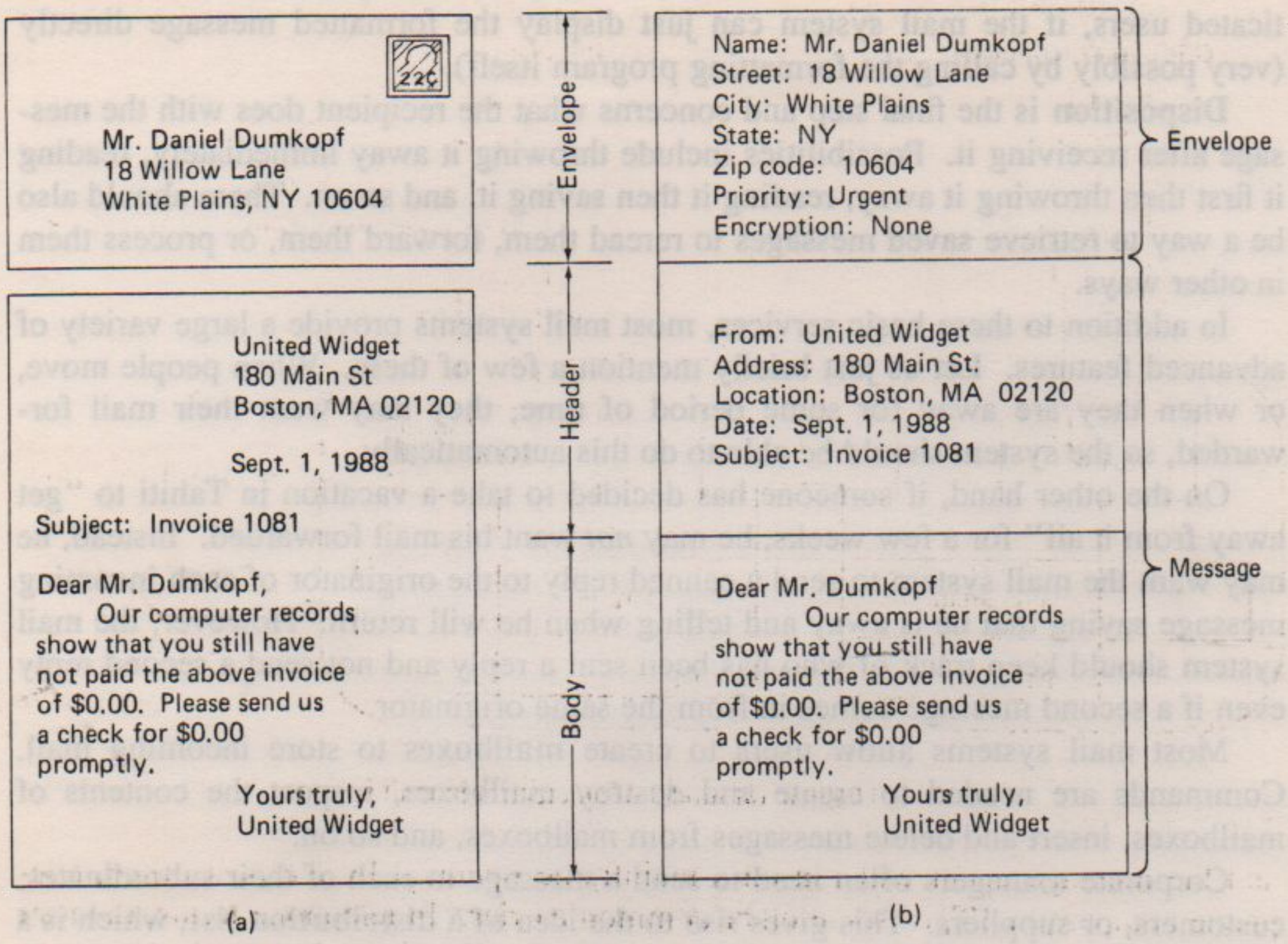


Fig. 9-10. Envelopes and messages. (a) Postal mail. (b) Electronic mail.

whether his message was delivered or not. In MOTIS, users can request that a message generate a reply only if it is delivered, only if it is not delivered, in all cases, or in no cases.

Probes are special test messages consisting of empty envelopes. The purpose of sending a probe is to find out if the destination is reachable. Furthermore, from the time and location stamps attached by all the intermediate machines along the way, the sender can also determine what route the message followed and how long each hop took.

Figure 9-11 shows the model used by MOTIS and most other electronic mail systems. The **user agent** is a program that provides the interface to the mail system. It allows users to compose, send, and receive mail, and manipulate mailboxes.

The **message transfer agent** accepts mail from user agents and sees to it that the mail is started on its way. The message transfer agent is the electronic post office. Just as with the postal system (known in e-mail circles as **snail mail**) a message may visit several (electronic) post offices before being delivered.

While it is possible for the user agent and the message transfer agent to reside on the same host computer, in the future this will be the exception rather than the rule. Typically, the user agent will run on a personal computer at home or work,

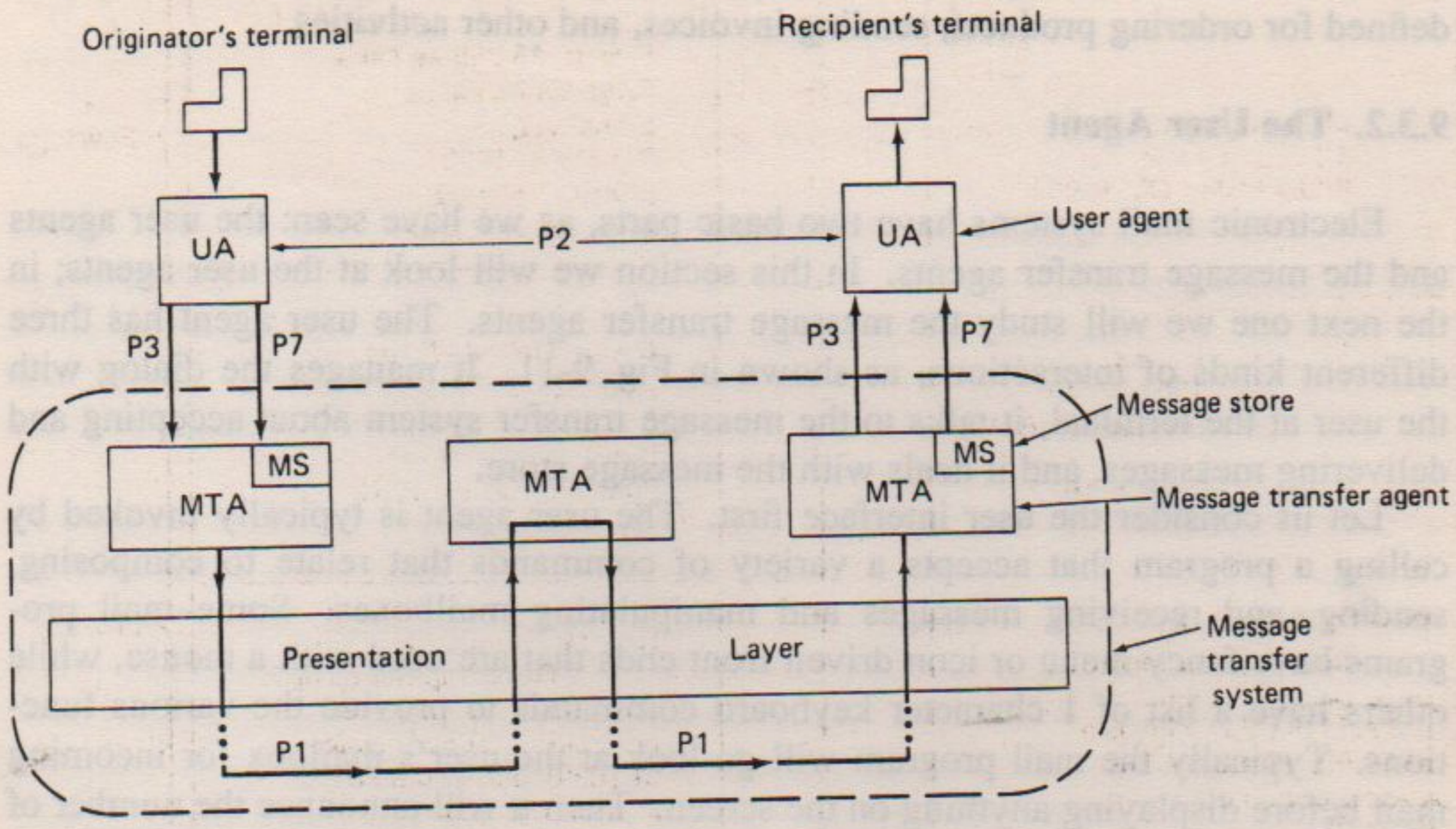


Fig. 9-11. The general model of an electronic mail system.

and the message agent will run on a mainframe operated by the post office, telephone company, PTT, or some other common carrier. In telephone terms, the user agent is the telephone and the message transfer agent is the carrier's end office.

Personal computers have a limited storage capacity and are furthermore only connected to the message transfer agent a short fraction of the day. This gives rise to the question of what to do with very large messages or messages that come in when the user agent is not logged in. The solution is to have the message transfer agent maintain electronic mailboxes for each user in an area called the **message store**. Incoming messages can be placed in these mailboxes until the user logs in to read them, delete them, move them to other mailboxes, selectively upload them, and so on.

If the user agent and message transfer agent run on separate machines, the interaction between them, *P3*, becomes a subject for standardization. OSI has defined service and protocol standards for it. Similarly, the interaction with the message store, *P7*, has also been standardized, as has *P1*, the protocol between the message transfer agents.

The collection of all message transfer agents is called the **message transfer system**. It is divided up into **administrative domains**. Different domains are operated by different authorities. Some domains are public, operated by a common carrier or PTT, whereas others are private, entirely local to a single company.

The (virtual) protocol between the user agents, *P2*, has also been standardized. In effect, *P2* is defined by the header and body of Fig. 9-10 whereas *P1* deals with the envelope. *P2* is known as **IPM (interpersonal messaging)**, and is intended for

human to human messages. As time goes on, no doubt other protocols will be defined for ordering products, sending invoices, and other activities.

9.3.2. The User Agent

Electronic mail systems have two basic parts, as we have seen: the user agents and the message transfer agents. In this section we will look at the user agents; in the next one we will study the message transfer agents. The user agent has three different kinds of interactions, as shown in Fig. 9-11. It manages the dialog with the user at the terminal, it talks to the message transfer system about accepting and delivering messages, and it deals with the message store.

Let us consider the user interface first. The user agent is typically invoked by calling a program that accepts a variety of commands that relate to composing, sending, and receiving messages and manipulating mailboxes. Some mail programs have fancy menu or icon driven front ends that are used with a mouse, while others have a list of 1 character keyboard commands to provide the various functions. Typically the mail program will go look at the user's mailbox for incoming mail before displaying anything on the screen. Then it will announce the number of messages in the mailbox and wait for a command.

As an example of how a user agent works, let us take a look at a typical mail scenario. After starting up the mail program, the user asks to see the messages in the mailbox used for incoming mail. A display like that of Fig. 9-12 then appears on the screen. Each line refers to one message. In this example, the mailbox contains eight messages.

#	Flags	Bytes	Sender	Subject
1	K	1030	Johan	Bug in MINIX file system
2	K F	2146	Ann L. Jones	Request for information
3	KA	7136	RVR	Comments about performance paper
4		3124	Erik	Improvement to the basic model
5		610	Henri	Meeting postponed until Tuesday
6		724	janlo	I think the solution is 0-0
7		3240	Peter Smith	Invitation to come see kangaroos
8		425	Edith	Don't forget to prepare your exam

Fig. 9-12. An example display of the contents of a mailbox.

Each display line contains several fields extracted from the envelope or header of the corresponding message. In a simple mail system, the choice of fields displayed is built into the program. In a more sophisticated system, the user can specify which fields are to be displayed by providing a **user profile**, a file describing the display format. In this example, the first field is the message number. The second field, *Flags*, can contain a *K*, meaning that the message is not new, but was read previously and kept in the mailbox; an *A*, meaning that the message has

already been answered; and/or an *F*, meaning that the message has been forwarded to someone else. Other flags are also possible.

The third field tells how long the message is and the fourth one tells who sent the message. Since this field is simply extracted from the message, this field may contain first names, full names, initials, login names, or whatever else the sender chooses to put in that field. Finally, the *Subject* field gives a brief summary of what the message is about. People who fail to include a *Subject* field often discover that answering their mail tends not to get the highest priority.

After the headers have been displayed, the user can perform any of the commands listed in Fig. 9-13. Some of the commands require a parameter. The # sign means that the number of a message (or perhaps several messages) is expected. Alternatively, the letter *a* can be used to mean all messages. The display of Fig. 9-12 was produced by the *h a* command.

Command	Parameter	Description
h	#	Display header(s) on screen
c		Display current header on screen
t	#	Type message(s) on screen
s	address	Send a message
f	#	Forward message(s)
a	#	Answer message(s)
d	#	Delete message(s)
u	#	Undelete previously deleted message(s)
m	#	Move message(s) to another mailbox
k	#	Keep message(s) after exiting
r	mailbox	Read a new mailbox
n		Go to next message and display it
b		Back up to previous message and display it
g	#	Go to a specific message but do not display it
e		Exit mail system and update mailbox

Fig. 9-13. Typical mail handling commands to the user agent.

Our example mail program (patterned after the *mmdf* program used on UNIX) keeps track of the current message. The *c* command prints the current message's header. The *t* command types (i.e., displays on the screen) the requested message or messages. Possible commands are *t 3*, to type message 3, *t 4–6*, to type messages 4 through 6, and *t a* to type them all.

The next group of three commands deals with sending messages rather than receiving them. The *s* command sends a message by calling an appropriate editor (e.g., specified in the user's profile) to allow the user to compose the message. Spelling, grammar, and diction checkers can see if the message is syntactically correct. Unfortunately, the current generation of mail programs do not have

checkers to see if the sender knows what he is talking about. When the message is finished, it is prepared for transmission to the message transfer agent.

The *f* command forwards a message from the mailbox, prompting for an address to send it to. The *a* command extracts the source address from the message to be answered and calls the editor to allow the user to compose the reply.

The next group of commands is for manipulating mailboxes. Users typically have one mailbox for each person with whom they correspond, in addition to the mailbox for incoming mail that we have already seen. The *d* command deletes a message from the mailbox, but the *u* command undoes the delete. (The message is not actually deleted until the mail program is exited.) The *m* command moves a message to another mailbox. This is the usual way to save important mail after reading it. The *k* command keeps the indicated message in the mailbox even after it is read. If a message is read but not explicitly kept, some default action is taken when the mail program is exited, such as moving it to a special default mailbox. Finally, the *r* command is used to finish up with the current mailbox and go read another one.

The *n*, *b*, and *g* commands are for moving about in the current mailbox. It is common for a user to read message 1, answer, move, or delete it, and then type *n* to get the next one. The value of this command is that the user does not have to keep track of where he is. It is possible to go backwards using *b* or to a given message with *g*.

Finally, the *e* command exits the mail program and makes whatever changes are required, such as deleting some messages and marking others as kept. This command overwrites the mailbox, replacing its contents.

It should be clear from this example that electronic mail has come a long way from the days when it was just file transfer. Sophisticated user agents make it possible to manage a large volume of correspondence easily. For people (such as the author) who receive and send thousands of messages a year, such tools are indispensable.

Let us now turn from the user interface to the protocol used between user agents (*P2* in Fig. 9-11). This protocol is, to a large extent, defined by the header fields included in each message. Some of the common header fields are shown in Fig. 9-14.

The *Originator* field contains the name of the person who actually sent the message. The *Authorizing users* field tells who gave authority to send it. In most cases these two will be identical, but they might differ when a secretary sends a message on behalf of the boss.

The next three fields determine who gets copies of the message. The *Primary recipients* are the people it is really intended for. Carbon copies are sent to the *Copy recipients*, often for administrative purposes. The *Blind copy recipients* also get copies, but their names are not included in the copies sent to the others. Thus the primary recipients get to know who received carbon copies, but not who received blind carbon copies. This feature is widely used in many companies.

Field	Description
Originator	Who actually sent the message
Authorizing users	On whose behalf was it sent
Primary recipients	To whom is the message addressed
Copy recipients	Who gets carbon copies
Blind copy recipients	Who gets carbon copies secretly
Reply recipients	To whom should the reply be sent
Reply time	By when the reply desired
Message id	Message identifier
In reply to	Message to which this is a reply
Obsoleted messages	Messages invalidated by this one
Related messages	Other messages relevant to this one
Subject	What is the message about
Importance	Message priority
Sensitivity	Public, company confidential, etc.
Expiry time	Time when message ceases to be valid

Fig. 9-14. Some header fields for interpersonal messages.

The next group of fields relate to the reply. The reply does not necessarily have to go to the person who sent the message, and there may be a deadline by which it is required.

Then comes a group containing message identifiers: of this message, of the message it is replying to, of previous messages superseded by this one, and of other relevant messages.

The *Subject* field summarizes the message. We saw how this field is used in Fig. 9-12. The *Importance* field can specify the message to be urgent or of some other priority. *Sensitivity* has to do with how secret the contents are and whether the sender wishes them disclosed to third parties.

Finally, the *Expiry date* field tells how long the message is valid. For example, someone might offer to buy or sell some goods for a certain price, but the offer is only valid until a certain date.

Although the body of the message is not as tightly structured as the header, it does have some structure. The body consists of a main part and optionally one or more attachments (appendices). In the future, many people will be receiving electronic mail at home via videotex, using their television sets as the display device. Generally, televisions only have enough resolution to display 40 characters of text per line. If the originator assumed that a line contains 80 characters, a problem arises.

One solution is to encode the body as a standard **simple formatted document**. In such a document, the text is composed of paragraphs, with blank lines between the paragraphs. The user agent on the recipient's end displays as many words per line as fit, preserving only the paragraph boundaries, not the line boundaries.

A more sophisticated solution is to have all messages contain standardized

typesetting commands of the sort used in *troff*, *scribe*, T_EX, and other **markup languages**. The user agent can then display sophisticated text, including mathematical and chemical formulas, tables, line drawings, and footnotes in a form suitable for the recipient's terminal or printer. Work is progressing in this area.

9.3.3. The Message Transfer Agent

The message transfer system is concerned with relaying the message from the originator to the recipient. This process is fundamentally different than setting up an application-layer association, as is done in FTAM. With FTAM, you cannot read a remote file if the file server is currently down. With mail, you *can* send a message to someone, even if that person is currently on vacation, or if his user agent is not currently logged in. Electronic mail is thus modeled on a store-and-forward system that works hop-by-hop, not end-to-end.

When a message comes in to a message transfer agent, the processing goes something like this. If the message is from a user agent, the syntax is checked for validity, and if found invalid, it is sent back with an explanation. If it is valid, a message identifier and time stamp are affixed, and it is then treated the same way as a message arriving from another message transfer agent.

The next step is to see if the recipient's user agent or mailbox is local. If so, the message can be delivered, queued for delivery, or stored in the mailbox. If necessary, a reply message confirming delivery is generated and sent back. If the recipient is not local, the message is forwarded to another message transfer agent. International mail may have to transit a gateway at each national boundary.

In most systems, a log of message agents that have handled the message is appended to the envelope. Not only does this make it easier to track down problems, but it also makes it possible to check for loops. If a message transfer agent receives a message containing itself in the log, it knows the message is looping, and has to take special measures to break the loop.

Delivery to a local user agent is not always trivial, because the originator and recipient may have different types of equipment. Possible message types include:

1. ASCII text.
2. Analog facsimile.
3. Digital facsimile.
4. Digitized voice.
5. Videotex.
6. Telex.
7. External (some other system).

If the recipient cannot directly accept the message type, the message transfer agent

can attempt to convert it before delivery. Not all conversions are feasible. For example, converting digitized voice to ASCII or facsimile is a nontrivial project. If the conversion cannot be done, the message cannot be delivered.

It will be many years before everyone has the necessary equipment to receive e-mail. In the meanwhile, conversions to other systems will be of great value. For example, when sending a message to someone not having any kind of terminal, the message could be sent electronically to the post office closest to the destination address. There it could be printed on a mechanical printer, laser printer, or facsimile device. Digitized voice could be spoken by a speech synthesizer and then recorded on an audio cassette. The final output could be hand delivered by the postal system.

Alternatively, if the recipient has a personal computer without e-mail capability, the message could be written to floppy disk and the disk physically delivered by the letter carrier. When this mixed-system scheme is not as elegant as an end-to-end electronic one, sending the message to the final post office electronically and then having the printout, audio cassette, or floppy disk sent special delivery, means that it should be possible to have a message entered from a terminal be delivered almost anywhere to the final destination in a few hours at most.

Although ISO has not standardized all the details of the store-and-forward operation between message transfer agents, it has adopted CCITT's general framework for what is called the **remote operation service**. This is none other than our friend the remote procedure call, only now buried inside the mail system, of all places. Four operations have been standardized:

1. Invoke remote operation on another computer.
2. Return result of a remotely invoked operation to the caller.
3. Return error message to the invoker.
4. Reject remote operation call as invalid (e.g., bad parameters).

The remote operation service could be used, for example, by one message transfer agent to execute the operation of entering a message in a processing queue on another one. Why CCITT suddenly embraced the connectionless model after fighting it tooth and nail for seven layers is not obvious. On the other hand, better late than never.

One of the key design issues for the message transfer agents is addressing. We have already seen one possible scheme in Chapter 5, namely, the NAME@DOMAIN system. This is the addressing system used in the ARPANET, ARPA Internet, USENET, and various other networks. Domains can have subdomains, so THOMPSON@CS.YALE.EDU refers to the Thompson in computer science at Yale, whereas THOMPSON@CHEM.YALE.EDU refers to the Thompson in chemistry.

This domain based naming system works fine with a user base of a million

people at universities, computer companies, and government installations. CCITT does not believe it would work nearly as well when e-mail reaches the entire population of the industrialized world. Consequently they have devised a radically different addressing scheme. However, we will defer the description of it until later in this chapter after we have covered the directory service, to which it is closely related.

The last aspect of the message transfer system that we will look at here is the envelope (see Fig. 9-10). Some of the fields are listed in Fig. 9-15. They are collected into four rough groups: addressing, delivery, conversion, and security. Although this list is not complete, it gives a reasonable feel for the envelope.

Field	Description
Originator's address	Mail address of the sender
Recipient's address	Mail address of the recipient
Alternate recipient allowed	Is redirection to someone else allowed
Alternate recipient	Second choice recipient
Message id	Message identification
Priority	Slow (cheap), normal, fast (expensive)
Originator report request	What reporting does originator want
MTA report request	What reporting does MTA want
Deferred delivery	Do not deliver before this time
Latest delivery	Do not deliver after this time
Content return	Should content be returned on nondelivery
Information type	Text, facsimile, digitized voice, etc.
Conversion prohibited	No conversion allowed
Lossy conversion prohibited	Only perfect conversion allowed
Explicit conversion	Conversion is known to be required
Encryption identification	Index into encryption key table
Content integrity check	Check sum on contents
Originator's signature	Digital signature
Message security label	Classified, secret, top secret, etc.
Proof of delivery	Recipient's signature

Fig. 9-15. Some of the envelope fields.

The need for the *Originator's address* and the *Recipient's address* are obvious. Less obvious are the next two fields. They deal with the possibility that the message can be delivered to the destination machine but not to the recipient, for example, because the name is spelled wrong or the recipient has moved. Some installations have a default rule that all undeliverable mail is delivered to the "postmaster," who looks at it and tries to figure out what to do with it. If the message is confidential, the originator can set the *Alternate recipient allowed* field to FALSE, so it will not be forwarded to the postmaster. It is also possible to specify a second choice recipient.

The second group concerns delivery. The first two items are self explanatory.

The *Originator's report request* allows the originator to specify that a reply message should be sent back if the message is delivered, if it is not delivered, always, or never. The reply specifies to whom it was actually delivered (the recipient, alternative, postmaster, or other), and when that happened. The *MTA report request* allows the message transfer agent to ask for its own reporting, distinct from the user reporting. This reporting can be more detailed, for example, requiring each message transfer agent along the route to sign and time stamp the envelope.

It is sometimes important that a message not be delivered too early or too late. For example, a company may wish to announce an important new product worldwide at the same moment. It may not want any of its announcement messages to be delivered too early, to avoid information leaking out. The *Deferred delivery time* field gives the earliest allowed delivery time. If the message actually arrives earlier, it must be held.

The reverse situation is when delivery after a certain time is not worth the trouble. This time is indicated by *Latest delivery time*.

Finally, *Content return request* can be set to insure that if the message cannot be delivered, that the entire message, not just the envelope and header, is returned. This feature is especially useful for people who send a lot of mail and do not keep copies of it, so if something comes back as undeliverable, they will know what it was.

The third group deals with conversion between ASCII text, teletex, facsimile, digitized voice, and other information types. If the recipient happens to have the wrong kind of device, sometimes a conversion can be made. However, conversions can result in lost information. For example; teletex terminals do not have the full ASCII character set. The more exotic symbols may have to be replaced by question marks or even dropped altogether. This group of fields gives the originator some control over the conversion process and lets him state whether a lossy conversion is better than no delivery at all.

The final group deals with security. It provides for message encryption, checksums on the contents (to allow message tampering to be detected), the originator's digital signature (to prevent originators from repudiating their messages), security codes, and the recipients digital signature (to prevent recipient's from denying delivery). We have discussed cryptographic algorithms to achieve these goals in the previous chapter. The exact choice of algorithm is up to the users. These fields merely provide a standardized place to put the information.

9.4. VIRTUAL TERMINALS

Terminals fall in three broad classes: scroll mode, page mode, and form mode. Scroll mode terminals are the simplest; form mode terminals are the most sophisticated. For each category, different problems are present and different approaches

are needed. We discuss each of the three classes in some detail below. Some additional information on this subject can be found in (Gilmore, 1987).

9.4.1. Scroll Mode Terminals

Scroll mode terminals do not have built-in microprocessors or any local editing capability. When a key is hit, the corresponding character is sent over the line (and possibly also displayed). When a character comes in over the line, it is just displayed. As new lines are displayed, the old ones just scroll upward. Most hardcopy terminals, as well as some CRT terminals are, of this type.

Despite their relative simplicity, scroll mode terminals can differ in many ways, for example, character set, line length, presence or absence of automatic echoing, and overprinting, not to mention the way carriage return, line feed, horizontal tab, vertical tab, backspace, form feed, and break are handled. Some terminals also have potential timing problems, such as the number of filler characters needed following tab and carriage return.

Because scroll mode terminals do not have any processing power, they cannot communicate with the network using any of the network's standard protocols. (A terminal with a built-in CPU can run protocol software internally.) To solve this problem, people normally buy a "black box" that is inserted between the terminal and the network. The black box speaks RS-232 to the terminal, and some standard protocol to the network. It is generally called a **PAD (Packet Assembler/Disassembler)** and it can also do various conversions.

The Packet Assembler/Disassembler

CCITT has defined standard PAD interfaces in its X.3, X.28, and X.29 recommendations (triple X). X.3 defines the PAD parameters, X.28 defines the terminal-PAD interface, and X.29 defines the PAD-computer (DTE) interface. The PAD is not exactly a virtual terminal, but it does not really fit into the OSI hierarchy very well anywhere else, either.

When a terminal initially establishes contact with a PAD, the human operator or the computer sets up certain parameters describing the conversation between the terminal and the PAD. The parameters are numbered and so are their option values. A typical command to the PAD might be: SET 1:0, meaning set parameter 1 to value 0. Other commands are available to establish and break connections, read out the values of the parameters, reset the line, and force an interrupt. The complete list is given in Fig. 9-16.

Parameter 1 has to do with whether or not the terminal operator can change PAD parameters in the middle of a session. Parameter 2 must be set according to the terminal type.

Parameters 3 and 4 are needed because some carriers charge per packet, not per character. If a user wishes to be frugal, he will only send full packets, since they

Parameter	Description	Allowed values
1	Can the terminal operator escape from data transfer mode to inspect or change the PAD parameters?	0 = No (escape prohibited) 1 = Yes (escape allowed)
2	Should the PAD echo characters back to the terminal?	0 = No 1 = Yes
3	Which characters should trigger the PAD into sending a partially full packet?	0 = Only send full packets 1 = Carriage return 126 = All control chars + DEL
4	How fast should the PAD time out and send a partially full packet?	0 = No time out 1–255 = Time in 50 ms ticks
5	Can the PAD (temporarily) prohibit the terminal from sending characters?	0 = No (Raw mode) 1 = Yes (Flow control)
6	Is the PAD allowed to send service signals to the terminal?	0 = No (suppress signals) 1 = Yes (deliver signals)
7	What should the PAD do upon receiving a break signal from the terminal?	0 = Nothing 1 = Interrupt 2 = Reset 4 = Send a host a control packet 8 = Escape to command mode 16 = Discard output
8	Should the PAD discard computer output intended for the terminal?	0 = No (Deliver) 1 = Yes (Discard)
9	How many fill characters should the PAD insert after outputting a carriage return to the terminal?	0 = None 1–7 = Number of fillers
10	Should the PAD automatically fold output to prevent line overflow?	0 = No 1–255 (yes) line length
11	Terminal speed (bps)	0 = 110 8 = 200 1 = 134.5 9 = 100 2 = 300 10 = 50
12	Can the terminal (temporarily) prohibit the PAD from sending it output?	0 = No 1 = Yes
13	Should the PAD insert line feed after carriage return?	0 = No 1–7 various conditions
14	Should PAD add filler after line feeds?	0 = No 1–7 = Yes (How many)
15	Does PAD allow editing	0 = No 1 = Yes
16	Select keystroke for character delete	0–127 = character
17	Select keystroke for line delete	0–127 = character
18	Select keystroke for line display	0–127 = character

Fig. 9-16. The PAD parameters (from CCITT standard X.3).

cost the same as partial packets. The trouble is, that to make intelligent use of an interactive terminal, the host must be notified every time a carriage return is typed. If the typed in characters just pile up in the PAD waiting for the packet to fill, the host will not react to them. Therefore options are provided to override the "send-only-full-packet" rule. Parameter 3 can be set to force a packet to be sent whenever a control character is typed. Parameter 4 can be set to force a packet to be sent whenever no input is received for a specified time interval. Depending on how the host has been programmed, it might be possible for the PAD to collect characters from several terminals and put them in the same packet to reduce costs.

Parameter 5 is used when the terminal also contains a paper tape reader or other relatively high-speed input device. When this parameter is set to 1, the PAD may start and stop the device to regulate the flow. The ASCII control characters DC1 and DC3 are used for on and off, respectively. Parameter 6 can be used to suppress certain status reporting from the PAD. This option is useful when nontechnical personnel (e.g., bank tellers) are using the terminal. Parameter 7 tells the PAD what to do when the user types a break character. The options may be combined. Parameter 8 can be set to cause the PAD to discard computer output (e.g., after receiving a break). Parameter 9 is needed because most hardcopy terminals require filler characters after a carriage return. Even some CRT terminals require filler characters above 4800 bps.

Parameter 10 tells the PAD the terminal line length, so that extra long output lines can be correctly folded. Parameter 11 gives the speed. Other speeds will probably be added in the future. Parameter 12 allows the user to temporarily shut the PAD up using DC1 and DC3, for example, to read the contents of the screen before it scrolls out of view.

Parameters 13 and 14 have to do with the fact that some terminals require a line feed after a carriage return or some filler characters after a line feed. These parameters allow the user to instruct the PAD to insert the necessary line feeds or filler characters.

Parameters 15 through 18 allow the user to edit text that has been typed in but not yet transmitted. If editing is enabled (parameter 15), the user can specify which characters will be used for deleting lines and characters and displaying the current line. CCITT may add new parameters to the PAD in the future, as the need arises.

9.4.2. Page Mode Terminals

Page mode terminals are typically CRT terminals that can display 25 lines of 80 characters each. The computer can move the cursor around the screen to modify selected portions of the display. These terminals have all the same problems as scroll mode terminals, plus a few more such as page length, cursor addressing, and the presence or absence of blinking, reverse video, color, and multiple intensities.

No two terminals have the same escape sequences for using all these fancy features. This makes writing screen editors (and other display-oriented software)

difficult. One widely-used solution, pioneered in Berkeley UNIX, is to define a virtual terminal consisting of the commands that most page mode terminals have.

When an editor starts up, it inquires about the terminal type, and then reads in the entry for the terminal from a data base called **termcap** (terminal capabilities). This entry gives the escape sequence required for each virtual command. As long as the software restricts itself to issuing virtual terminal commands, it will run on any terminal having a termcap entry.

Figure 9-17 shows a few of the more common termcap codes. The first group consists of Booleans that are present in the termcap entry if the associated statement is true, and absent if it is false.

The second group provides numerical values, such as the size of the screen. Armed with these numbers, the software knows how many lines of how many characters it can display.

The third group consists of escape sequences for clearing parts of the screen, moving the cursor, entering various special modes, and so on. If a particular code is not present in the termcap data base, the terminal does not have that capability, and the software must avoid it. In many cases missing capabilities can be handled with alternative commands.

For example, if a terminal does not have the *cl* code, but does have *cm* and *cd*, the screen can be cleared by first moving the cursor to the upper left-hand corner, and then clearing the screen from the cursor to the end. Similarly, an editor that wants to have error messages stand out, might prefer reverse video, but settle for bold if reverse video is not available, and use blinking as a last resort.

The fourth group in Fig. 9-17 is an attempt to deal with keyboard input. It is possible to define the escape sequences generated by the arrows and some other keys that move the cursor. In this way, users can move the cursor around the screen with the arrows in a terminal independent manner. The ability of termcap to deal with input is limited, however.

9.4.3. Form Mode Terminals

The most sophisticated terminals are those with built-in microprocessors. In applications such as banking and airline reservations, the computer can download a form to the terminal, with some of the fields being read-only and containing information, with the other fields to be filled in by keyboard input. The microprocessor can allow local editing, macros, and other facilities. These terminals are often called **form mode terminals**.

When the form has been filled in, the microprocessor can run a quick syntax check on it (e.g., bank accounts should be eight decimal digits, with the sum of the digits modulo 9 being 0). If the form is syntactically correct, the modified portion can be uploaded across the network back to the computer. It should be obvious that the termcap model is not adequate for these applications. A radically different virtual terminal model is needed.

Code	Type	Description
bs	Boolean	Terminal can backspace with CTRL-H
hc	Boolean	Terminal prints hard copy
nc	Boolean	Terminal is a CRT but is not able to scroll
os	Boolean	Terminal can overstrike (multiple characters at same position)
ul	Boolean	Terminal can underline text
pt	Boolean	Terminal has hardware tabs
co	Integer	Number of columns in a line
li	Integer	Number of lines on the screen or on a page
dB	Integer	Number of millisec of delay needed for backspace
dC	Integer	Number of millisec of delay needed for carriage return
dN	Integer	Number of millisec of delay needed for line feed
dT	Integer	Number of millisec of delay needed for tab
cd	String	Clear screen from cursor to end of display
ce	String	Clear screen from cursor to end of line
cl	String	Clear entire screen
cm	String	Cursor motion
ct	String	Clear all tab stops
dc	String	Delete character under cursor
dl	String	Delete line containing cursor
ff	String	Go to top of next page (hardcopy terminals)
ic	String	Insert one character at cursor
is	String	String used to initialize terminal (and perhaps set tabs)
mb	String	Enter blinking mode
md	String	Enter bold (extra-bright) mode
us	String	Enter underscore mode
mh	String	Enter dim (half-bright) mode
so	String	Enter reverse video mode
me	String	Enter normal mode (no blinking, bold, dim, reverse video, etc.)
pc	String	Character to use for padding after tab, line feed, etc.
sf	String	Scroll screen forwards
sr	String	Scroll screen backwards
kl	String	Escape sequence sent by left arrow key
kr	String	Escape sequence sent by right arrow key
ku	String	Escape sequence sent by up arrow key
kd	String	Escape sequence sent by down arrow key
kh	String	Escape sequence sent by home key
kb	String	Escape sequence sent by backspace key

Fig. 9-17. A few of the more common entries in the termcap data base.

An alternative model that is well suited for sophisticated display-oriented applications is based on the use of shared data structures. In this model, the virtual terminal software maintains an abstract representation of the display image internally. This representation can be read and updated by both the terminal user and the application program running somewhere on the network. The virtual terminal software has the responsibility for making sure that the screen image is updated whenever its abstract representation is modified.

Actually, this model has two major variants. In the first one, shown in Fig. 9-18(a), there is a single abstract data structure representing the screen image. Two identical copies of it are maintained, one by the virtual terminal software running on (or near) the terminal's microprocessor and the other by the virtual terminal software running near the application program on a distant host computer.

The person at the terminal can modify the microprocessor's copy of the abstract data structure by typing on the keyboard (the arc labeled 1 in Fig. 9-18). These changes are visible on the screen because the screen display is driven by the local data structure. The microprocessor then sends virtual terminal commands to the distant host over the network using the virtual terminal protocol (labeled 2 in Fig. 9-18). These PDUs cause the remote copy of the data structure to be brought up to date with the local one. The modified data structure can be read by the application program using appropriate commands (labeled 3 in Fig. 9-18).

Similarly, the application program can modify its copy of the data structure, which causes PDUs to be sent to the microprocessor to update the terminal's data structure and hence its display. These steps are shown in Fig. 9-18 as 4, 5, and 6.

To prevent both sides from trying to modify the data structure simultaneously, a token is used to control update access. Either the computer or the terminal may hold the token, and only the token holder may update the data structure. The token may be requested and passed back and forth, analogously to the tokens in the session layer, although the virtual terminal token is unrelated to those tokens.

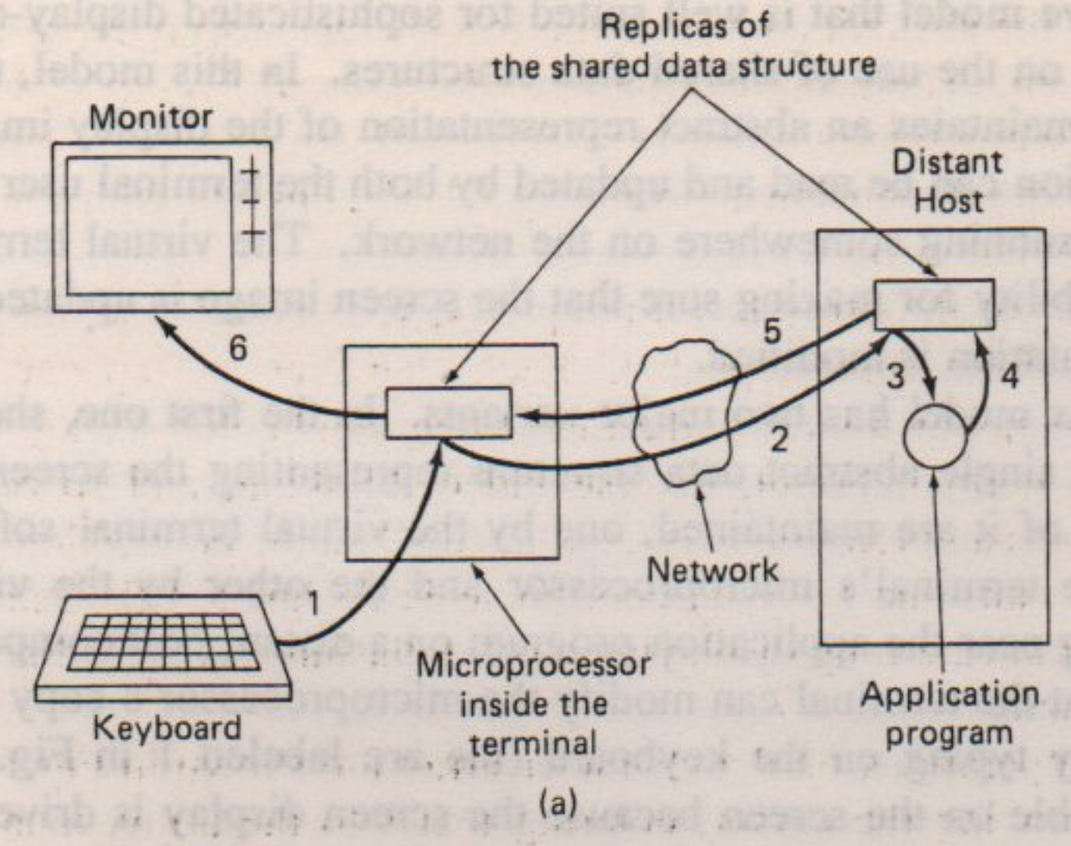
A consequence of the use of the token to control access to the data structure is that the terminal user may only type when the microprocessor holds the token. One could imagine equipping each terminal with two lights, along with a small sign saying:

Dear terminal user:
You may type when the green light is on, but please refrain from typing when the red light is on. The penalty for disobedience is that your typed input will be discarded. Thank you for your cooperation.

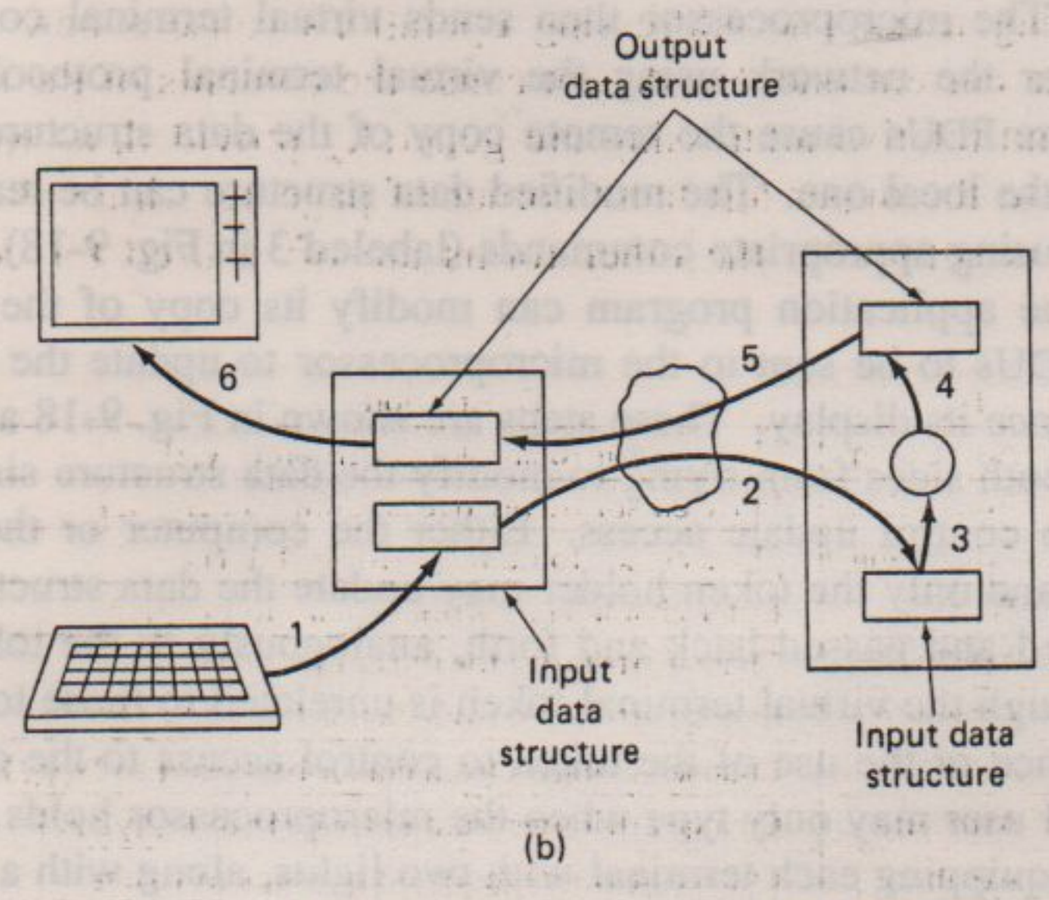
The terminal manager

(Actually, the terminal could buffer input without acting on it until the token arrived; nevertheless, the characters could not be echoed to the screen immediately.)

When this **synchronous** model was first proposed, not all users were equally enthusiastic about it. It did not take long for an alternative model to be proposed,



(a)



(b)

Fig. 9-18. Virtual terminal software using a shared data structure. (a) Synchronous model. (b) Asynchronous model.

namely, the **asynchronous** model of Fig. 9-18(b). The asynchronous model consists of two independent monologs, rather than a single dialog. Each end of the connection has a data structure for input and a second one for output. Changing your own output data structure affects your partner's input data structure, but it does not affect your input data structure or his output data structure.

In the asynchronous model, each copy of the data structure has a single reader and a single writer, so simultaneous write conflicts cannot occur. The price paid is additional storage and more complexity, although this is partly offset by not needing a token. All of what follows applies equally well to both models.

The heart of the abstract data structure is a character array representing the contents of the screen. The array can be one dimensional (to model intelligent typewriters with only one line of editable text), two dimensional (to model simple CRT terminals), or three dimensional (to model CRT terminals with multiple pages of text or multiple windows).

Each element of the array has a unique (x, y, z) coordinate, and can contain a single character from some alphabet. Each character can appear in any one of several **renditions**, which include the font, foreground color, background color, and emphasis (dim, normal, bright, reverse video, blinking, and so on). The set of available alphabets and renditions is negotiated when the connection is established.

At every instant, one (x, y, z) coordinate is designated the **current position**. Most terminals indicate it by displaying a solid, flashing, or underline **cursor** at that position on the screen.

Some basic operations on the character array are shown in Fig. 9-19. They are of four types: moving the cursor, entering text into the array, manipulating attributes, and erasing text. The addressing operations cause a new character to be designated as the current one, but do not cause the contents of the array to change. Both absolute and relative addressing are generally provided, as well as an operation that moves to the upper left-hand corner of page 1. Although in theory absolute addressing is sufficient, the other forms are often used for convenience and efficiency.

Type	Description
Addressing	Move cursor to absolute position (x,y,z)
Addressing	Move cursor by relative amount (Δx,Δy,Δz)
Addressing	Move cursor to home position (1,1,1)
Text	Enter characters starting at current position
Attribute	Change attributes for the entire array
Attribute	Change attributes from cursor forward to (x,y,z)
Attribute	Change attributes from cursor backward to (x,y,z)
Attribute	Change attributes from (x1,y1,z1) to (x2,y2,z2)
Attribute	Change attribute mode on subsequent writes
Erase x	Erase this line from start to cursor
Erase x	Erase this line from cursor to end
Erase x	Erase this line from start to end
Erase y	Erase this page from start to cursor
Erase y	Erase this page from cursor to end
Erase y	Erase this page from start to end
Erase z	Erase from start of first page to cursor
Erase z	Erase from cursor to end of last page
Erase z	Erase from start of first page to end of last page

Fig. 9-19. Basic operations on the virtual terminal character array.

One text operation is generally sufficient. It enters new characters into the

array starting at the current position and advances the cursor to the first position beyond the newly entered text. Several attribute operations are commonly provided. Some change the attributes of characters already in the array, but one is needed to change the current attribute, which is used on new characters. Once the current attribute has been set, it remains in effect until changed.

Various commands are available for erasing characters from the current line, page, or multiple pages. Erasing a character causes it to appear on the screen as a blank space. The characters to the right of an erased character do *not* get shifted to the left.

Given only the basic commands, scrolling the screen is difficult: it would require copying an entire subarray almost the full size of the screen. One solution to this problem is to make the *y*-dimension of the array infinite and to introduce the concept of a fixed-size **update window**. Typically, the update window has the width of the screen (e.g., 80 characters) and a height equal to the number of lines on the screen (e.g., 25 lines).

As an example, several hours into a long session, the current *y* position might be 5760, and the update window might extend from 5736 to 5760. As soon as a character was written to an array element on line 5761, the update window would be move automatically to the range 5737 to 5761. Only those characters within the update window are visible on the screen, and only those characters within the update window may be modified.

More advanced virtual terminal systems also provide direct support for forms. Form definitions can be downloaded into the terminal. A typical form consists of multiple fields, each field either containing read-only text, or blank space to be filled in by the terminal user according to some grammar associated with that field. Commands similar to the basic ones are provided: the cursor can be moved absolutely or relatively to a given field, text can be entered into a field, the attributes for certain fields can be changed, and fields can be erased. An example form is shown in Fig. 9-20. This form has nine fields, five of which are read-only (including the title), and four of which are to be filled in from the terminal.

Membership Application - Pigeon Fancier's Society

Name: Telephone:

Address: # of Pigeons:

Fig. 9-20. A typical form.

In general, virtual terminal definitions do not specify *when* updates have to be transmitted to the other side. It would be very inefficient to transmit each character typed on the terminal as a separate PDU, although there are some applications that demand this. In many cases it is better to collect an entire line or field, and transmit a single PDU at the end.

In particular, consider what happens when a user enters a line or field, and then part way through notices a typing error, erases the last few characters typed, and then retypes them. Should the virtual terminal software transmit all the characters typed, including the wrong ones, the backspaces, and the corrections? Probably not. Under normal circumstances it is sufficient to transmit the net effect of all the typing, namely the corrected line or field.

Similarly, it might happen that a terminal user decides to fill in the fields of a form in an order different from the top to bottom order. It may be possible to transmit a command sequence involving cursor motion and text that is shorter than what was actually typed, but has the same net effect. Many virtual terminal protocols allow each side to send any legal sequence of commands that achieves the same net effect as the commands that were actually typed in. This optimization is called **net-effecting**.

In general, the issue of how long the microprocessor should collect keystrokes before transmitting them or their net effect is called **delivery control**. The more it collects, the larger the chance that some optimization can be done. However, if it decides to collect several lines, and each line is a command to the computer to do something, then the terminal user will be surprised that nothing happens after the first line has been entered (because nothing has been sent yet).

Virtual terminal models can support various kinds of delivery control. In the simplest model, the virtual terminal software itself decides how much to buffer and when to send. One step up is to have a rule that all the net effect of all accumulated input is transmitted when certain characters (e.g., carriage return, line feed, or form feed) are input. Another rule is that no more than some number of characters will be buffered. Yet another possibility is not to allow input to accumulate for more than some specified time interval.

A more sophisticated form of delivery control is to have a primitive command *FLUSH* that sends the accumulated input. Data are never sent except when this primitive is executed. This scheme gives the user complete control over delivery. It is typically used on the computer side rather than on the terminal side.

One last issue that virtual terminals must address is **attention handling**. All systems have some key that the terminal user can hit to interrupt the current command or program. Sometimes it is the break hit; sometimes it is DEL or rubout. In any event, the virtual terminal software must deal with this event in a special way to allow this signal to overtake any data already in the pipe. One way is to use expedited data.

Suppose a user has asked the computer to list a file, and then after seeing the first few lines, has decided to kill the listing by hitting DEL. It would probably be a good idea for the terminal to start discarding output instantly, to give the user the feeling that the DEL has worked. However, the terminal is then faced with the decision of when to stop discarding output.

In particular, if the computer reacts to the DEL by outputting some kind of response or prompt, these characters must not be discarded. For this reason, the

DEL must be acknowledged in such a way that the terminal can tell which characters were output before the DEL (and must be discarded) and which were output after it (and must be displayed). If the computer acknowledges the DEL with a special character or character sequence in-band (thus not using expedited data), the terminal will be able to tell when to stop discarding (unless the special character happens to occur within the output to be discarded, in which case character stuffing is needed). The protocol has to be robust enough to work with impatient users who hit DEL three or four times in rapid succession, before even the first one has arrived at the computer.

9.5. OTHER APPLICATIONS

Numerous other applications are being developed and standardized. Some of them are for specific industries, such as banking, but others are general purpose. In this section we will take a quick look at a few general-purpose applications.

9.5.1. Directory Service

Services, SAPs, telephones, telex machines and other addressable entities found in telecommunication networks always have long numeric identifiers to make sure they are unambiguous. Since people tend to forget them, telephone books and directory services are provided to map names onto numbers. In the long run, these telephone books and information operators will be replaced by online directory services for looking up telephone numbers, network addresses, and so on. In France, most telephone subscribers already have a PTT terminal for this purpose. To prevent chaos, ISO and CCITT are developing standards for these directory services. In this section, we will provide an outline of this work.

At first glance, it might seem adequate to simply have an alphabetical list of service and subscriber names and their numbers, but different countries have different ways of doing things, so a fairly general scheme is needed. As an example, telephone books everywhere are sorted using the last name, although precisely what constitutes a last name varies from country to country (just ask John von Neumann, Anwar el-Sadat, or Willem de Kooning). In the U.S., all the people with with the same last name are sorted using their first names as the secondary key, whereas in Holland the street address is the secondary key. In some Scandinavian countries, a user-supplied string, such as an occupation or even an academic degree is the secondary sort key.

The basic idea of the OSI directory service is to allow users to look up names based on **attributes**. For example, one could ask the number of a person named Hudson working in IBM's sales office in Los Angeles. To accommodate the conventions of different countries as well as private directory services (company

phonebooks), the directory service consists of a hierarchy as shown in Fig. 9-21. Here, the top level directory contains entries of the form "C = country," although other hierarchies are also possible.

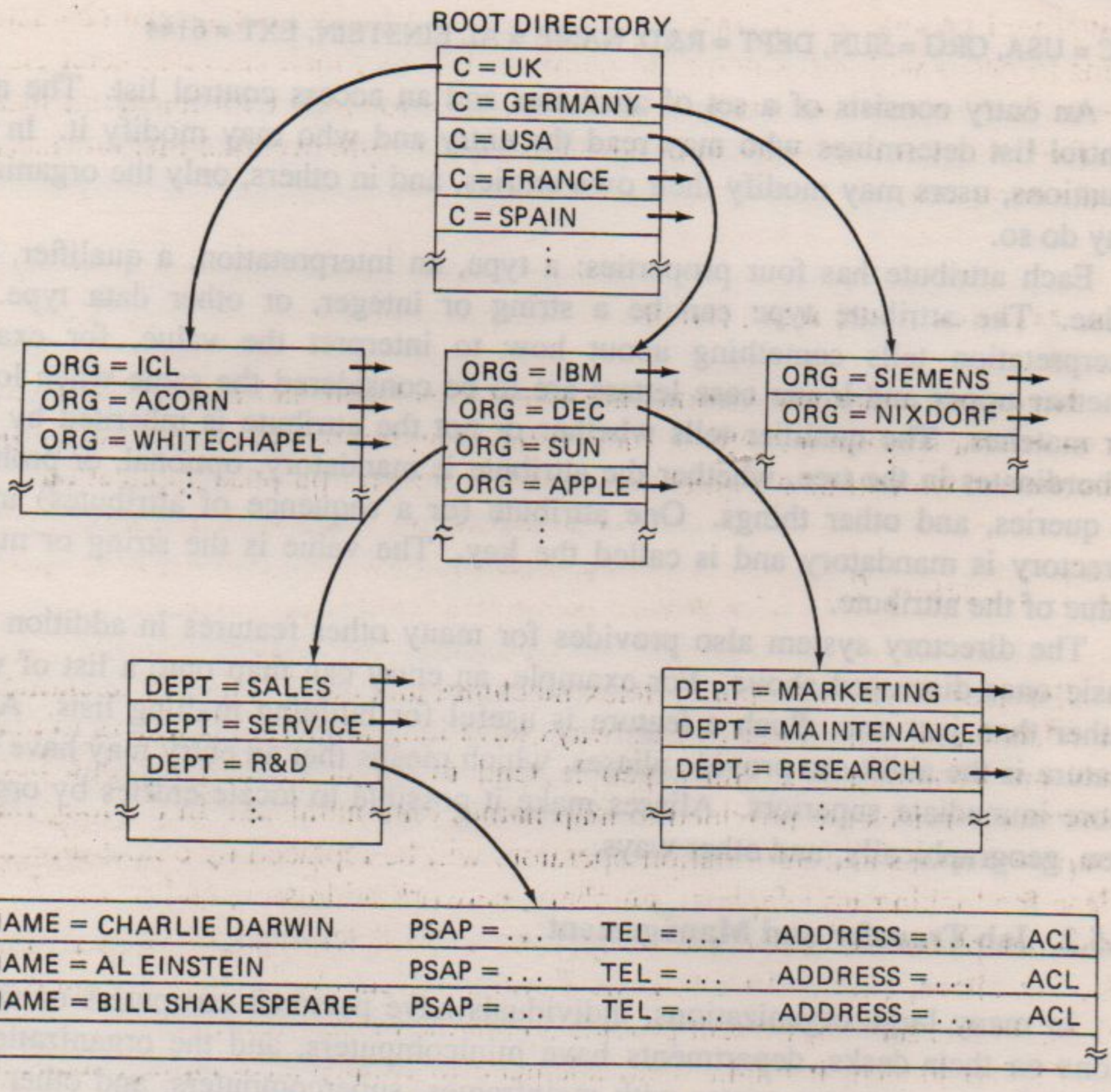


Fig. 9-21. An example directory hierarchy.

One level down, we have organization identifiers, denoted by "ORG = organization name." Below that we have departments, and finally at the bottom, entries for people (or services, machines, printers, etc). Higher level attributes can be inherited, so all entries below "ORG = NIXDORF" would act as though they also had the attribute "C = GERMANY."

To look up an entry, a user provides a set of attributes, such as

C = USA, ORG = SUN, DEPT = R&D, NAME = AL EINSTEIN

and a specification of which attributes of the entry are desired, such as the presentation address, telephone number, or street address. If the specification is unambiguous, the desired value is returned. If there is potential ambiguity and the user

knows some piece of information that can resolve it, that information can be used. For example, if there are two Al Einsteins in R&D and the user wants the PSAP address of the one whose telephone extension is 6144, the query could be formulated as

C = USA, ORG = SUN, DEPT = R&D, NAME = AL EINSTEIN, EXT = 6144

An entry consists of a set of attributes and an access control list. The access control list determines who may read the entry and who may modify it. In some situations, users may modify their own entries, and in others, only the organization may do so.

Each attribute has four properties: a type, an interpretation, a qualifier, and a value. The attribute type can be a string or integer, or other data type. The interpretation tells something about how to interpret the value, for example, whether upper and lower case letters are to be considered the same when looking for matches. The qualifier tells whether or not the attribute is inherited by all its subordinates in the tree, whether the attribute is mandatory, optional, or prohibited in queries, and other things. One attribute (or a sequence of attributes) in each directory is mandatory and is called the **key.** The value is the string or numeric value of the attribute.

The directory system also provides for many other features in addition to the basic ones discussed above. For example, an entry can map onto a list of values, rather than just one. Such a feature is useful for building mailing lists. Another feature is the ability to provide aliases, which means that an entry may have two or more immediate superiors. Aliases make it possible to locate entries by organization, geographically, and other ways.

9.5.2. Job Transfer and Management

In many large organizations, individuals have personal computers or workstations on their desks, departments have minicomputers, and the organization as a whole has a computer center with mainframes, supercomputers, and other expensive equipment. It frequently occurs that an individual prepares some work on his personal computer that must be run on a mainframe using files located on his department's minicomputer with the results to be sent back to the personal computer. The application that manages this kind of remote job entry is called **JTM** (**Job Transfer and Management**).

A typical scenario occurs when a person submits a **work specification** telling what is to be done, where it is to be done, where the input files come from, and where the output files go to. The JTM software must arrange that the program and all its input files end up on the machine where the job will be executed. Work specifications may consist of multiple jobs, so this entire process may have to be repeated several times.

One way is to send a request to each machine holding a file, and direct that

machine to send the file to the initiator's machine (e.g., using the standard file transfer protocol). When all the files and programs have been collected in this way, they are shipped to the execution machine.

A second way is for the initiating machine to direct the other machines to send the files directly to the execution machine, for example, to put them in a directory just created for that purpose.

A third way is to have the work specification move to one of the machine's holding input files, pick up the needed files, and then move on to the next machine. This migration continues until all the files have been collected, at which time the whole party moves on to the execution machine.

It is important to realize that JTM knows nothing about the contents of files, job control languages, or the nature of the processing. Its function is to see to it that input files are collected, programs are submitted for execution, and result files are deposited where they should be. It knows as little about the nature of the processing as FTAM does about the files it transfers.

As a concrete example of how execution could be requested without JTM knowing anything about how it works, imagine a computer with a directory called *InputQueue* in which processes can create subdirectories. Each subdirectory contains data files and a special file called *run* consisting of a shell script or job control language program. Whenever it is idle, the computer hunts around in *InputQueue* looking for a subdirectory not yet processed, and executes its *run* file.

As a job executes, it may create subjobs that are also submitted to JTM for remote execution. These subjobs may themselves require files either from the local machine or remote ones. In this manner, a single initial request may ultimately generate a tree of subjobs.

To design a JTM service, several problems must be solved. For one thing, every system involved in a job must have a unique name, so that there is no ambiguity. Telling JTM to run a job on the VAX is not very helpful if there are 50 VAXes around. If jobs may move over public networks, each system needs a worldwide unique name.

A second issue is authentication. When JTM requests an input file on behalf of a user, how does the file system holding the file know if the user has permission to access the requested file? How does the machine that eventually runs the job know who to charge for the CPU time? These are very interesting issues that can only be solved in a definitive way using cryptographic techniques such as digital signatures. In practice, the authentication methods used, such as passwords or trusting everyone are weak.

Finally, in addition to managing file and job movement, JTM is responsible for monitoring progress and reporting it. When complex jobs have been submitted, users are frequently interested in knowing how far along they are (e.g., has the initial job and all its files even reached the execution machine yet)? JTM can provide progress reports by sending report files back to the initiator after designated steps have been completed.

9.5.3. Picture Storage and Transfer

As high resolution displays become more common, there will be a corresponding increase in the number of people and applications interested in transmitting pictures as if they were text files. For this reason, a considerable amount of work has been done on standardizing picture representations so that pictures can be transferred between different locations, different applications, and different vendor's equipment. In this section we will briefly discuss this work.

Before getting started, we should make it clear that we are discussing the storage and transmission of line drawings, not photographs. Typical applications are electrical schematics, VLSI circuits, and architectural blueprints. All the figures in this book are line drawings.

Figure 9-22 shows the basic elements of a picture file. A **polyline** is a sequence of one or more line segments specified by a sequence of (x, y) pairs.

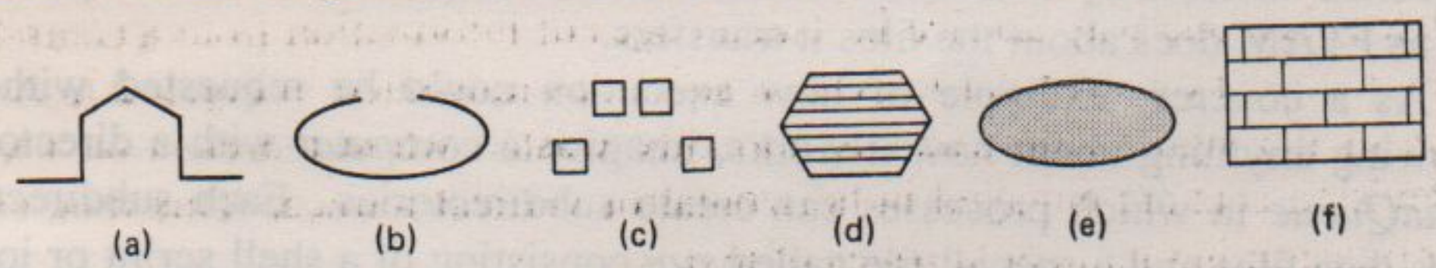


Fig. 9-22. (a) A polyline. (b) An elliptical arc. (c) A polymarker. (d) A filled polygon. (e) A filled ellipse. (f) A cell array.

An **elliptical arc** is a portion of an ellipse. It can be described by giving the coordinates of its foci, the length of the major axis, and the starting and ending angles of the arc. A circle is a special case of an ellipse, with both foci at the same position.

A **polymarker** is a sequence of points, like a polyline, except that they are not connected. The items drawn at each coordinate can be points, squares, circles, or other geometric figures.

A **filled polygon** is a closed figure formed by specifying a polyline that returns to its starting point. The polygon can be filled with a solid color, hatching, lines, dots, or a variety of other textures.

A **filled ellipse** is like a filled polygon, except that it is an ellipse that is filled, rather than a polygon. It is also possible to fill a portion of an ellipse, either as in pie diagrams or by drawing a chord and removing one part.

A **cell array** is a collection of rectangles filled with some pattern. Cells can be combined to give complex pictures.

In addition to these geometric elements, pictures may contain text in one or more alphabets at various positions and orientations.

Each of the picture elements has a variety of attributes that allow a wider variety of pictures to be drawn. For all the line segments, one can specify the type of line (solid, dashed, dotted), the color, and the thickness. Similarly, for polymarkers, the type, color, size, thickness, and orientation of the markers can be

specified. For filled figures and cells, the shape and texture must be given. Finally, for text, the fonts, sizes, and colors of the characters must be specified.

With these basics in mind, it should be clear how pictures can be represented as files. A picture file is a sequence of records, each record describing one picture element. Each element starts with a type field, followed by a list of its coordinates and other parameters. In this way, any line drawing can be reduced to a sequence of integers and characters for specifying its types, coordinates, angles, attributes and text. In effect, the picture description is like an assembly language program for an abstract drawing engine.

9.5.4. Teletext and Videotex

Teletext and Videotex are two new information-age services that will soon be widespread among home and business users. **Teletext** (not to be confused with teletex) is a one-way service in which pages of information from a central data base are broadcast on a regular television signal, as illustrated in Fig. 9-23. A small portion of the signal bandwidth is normally wasted while the electron beams in the television sets are engaged in horizontal or vertical retrace. This time can be used for transmitting numbered pages of text.

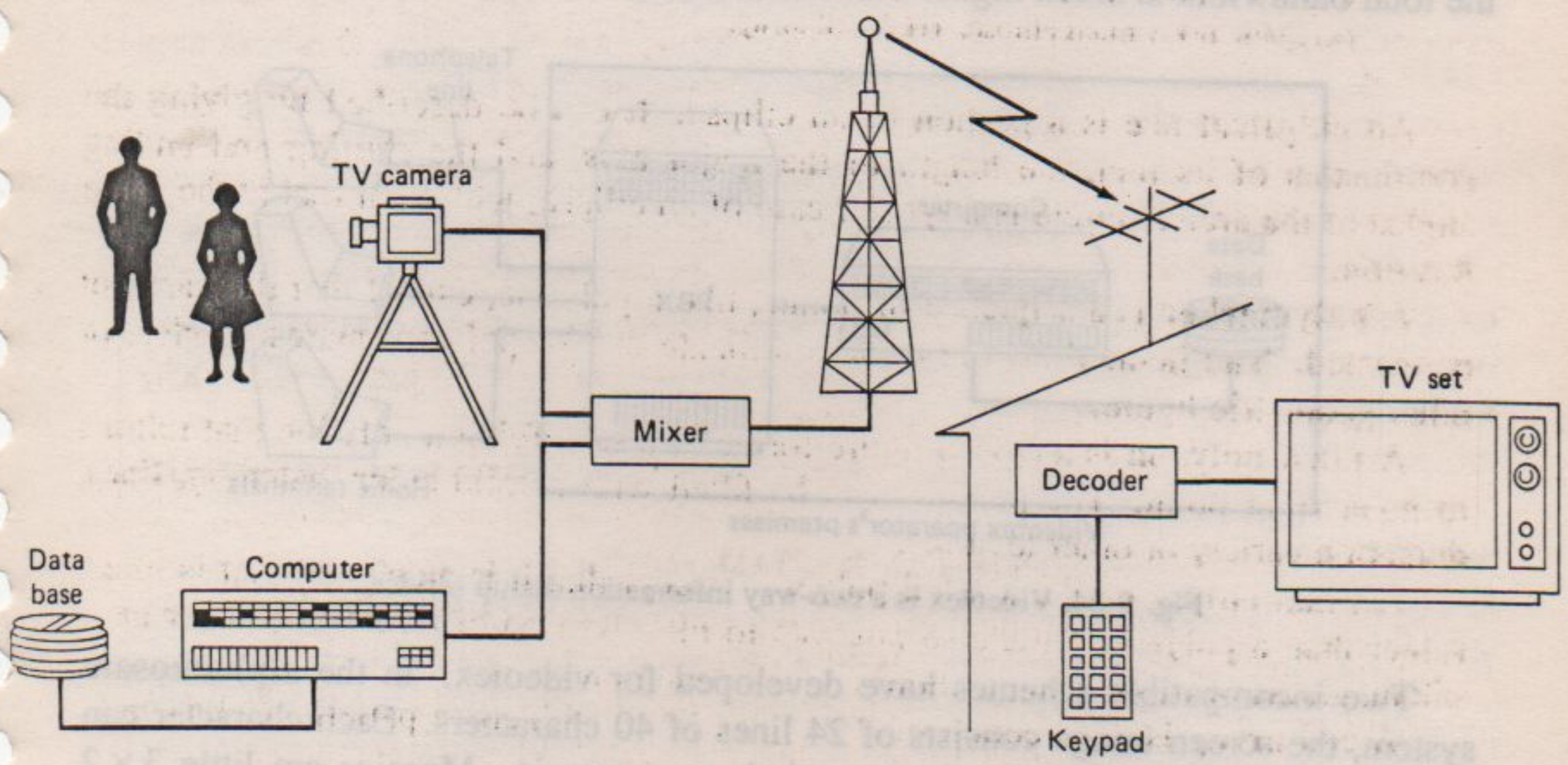


Fig. 9-23. Teletext is a one-way television-based information service.

If, for example, the bandwidth available for teletext is 8 kbytes/sec, and pages consist of 20 lines of 40 characters, the service can transmit 10 different pages in one second. A typical system might transmit 100 different pages in a 10-sec interval, and then repeat the pattern over and over every 10 sec. The contents of a given page may be different on successive passes as the data base is updated.

To use the system, the user types a page number on a keypad attached to the teletext decoder. The next time that page is transmitted (and every subsequent time, as well), it is plucked out of the air and displayed on the screen. With a 10-sec cycle time, there will be an average delay of 5 sec before the page appears. Thus there is a trade-off between the information capacity of the system (number of pages in the cycle) and the response time.

Teletext is most suitable for simple information in which many people are potentially interested. Uses include: weather forecasts, stock market reports, news, recipes, product information, and advertising. A simple file transfer protocol is all that is needed in the application layer.

Videotex is a more sophisticated two-way service using telephone lines. In effect, the home user logs into a large time-sharing system operated by the provider of the videotex service, as shown in Fig. 9-24. In many cases, the "terminal" is a keyboard that generates a video signal, so a standard color television set can be used as the display. Videotex is an interactive service in which users can request specific information. Unlike the teletext data base, which is restricted by the number of pages that can be broadcast in a few seconds, the videotex data base can be arbitrarily large because information is only transmitted when someone wants it. Furthermore, since each user has a dedicated channel into the videotex computer, the total bandwidth is much higher than for teletext.

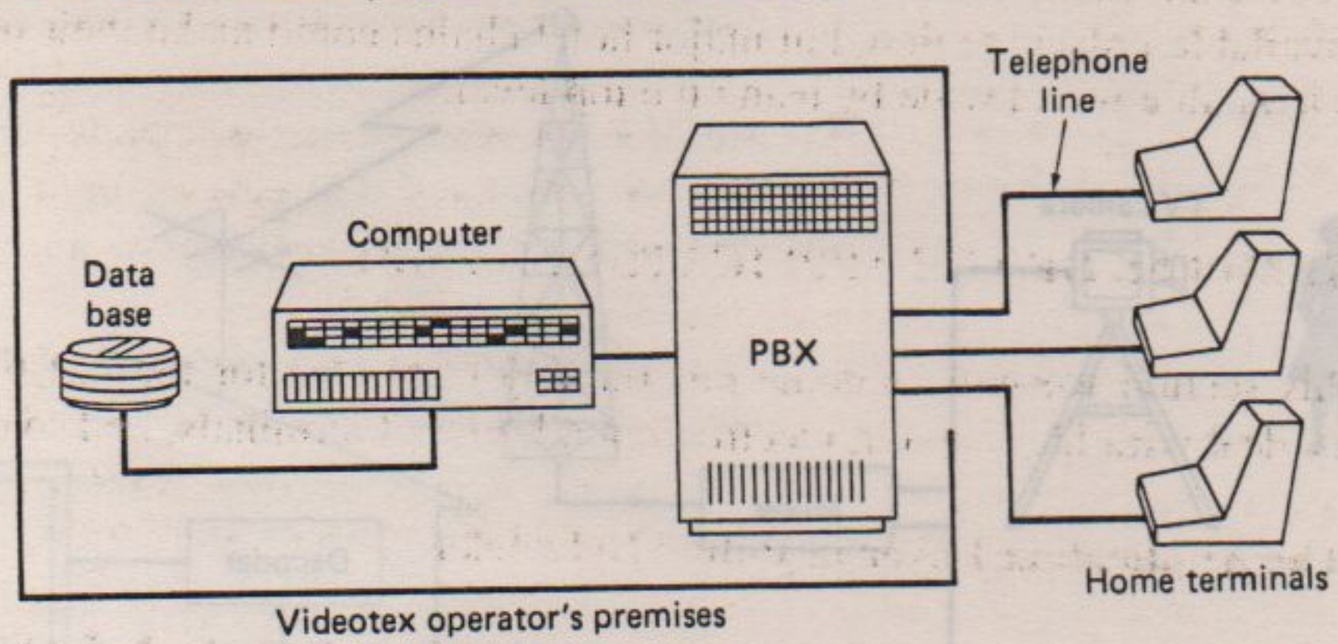


Fig. 9-24. Videotex is a two-way information dialup service.

Two incompatible schemes have developed for videotex. In the **alphamosaic** system, the screen image consists of 24 lines of 40 characters. Each character can be an ASCII character, a graphics symbol, or a mosaic. Mosaics are little 3×2 matrices. Using mosaics transforms the television into a 72×80 bit map display. This is not exactly high resolution, but is good enough for simple graphics.

In the **alphageometric** system, graphics are generated by sending picture drawing commands of the type described in the previous section (polyline, ellipse, and so on). This system gives good results even with low-speed modems, but it requires more hardware and is thus more expensive.

Videotex has two potential audiences: home users and business users. Home users may be interested in information services, such as airline, train and bus schedules, movie listings, restaurant menus, yellow pages, library card catalogs, news, sports, weather, and stock prices. They may also want to use the system in an interactive way, for home banking, catalog shopping, buying stocks, accessing electronic bulletin boards, and making airline, hotel, restaurant, or theater reservations. Sending and receiving electronic mail is also a possibility.

Corporations may use some of these services too (e.g., airline and hotel reservations). Furthermore, they may also want to use videotex to make certain information easily available to all employees nationwide. Some videotex operators offer closed user groups to cater to this audience. With closed user groups, certain information is restricted to members of the group. However, if the expected use is heavy, it may be cheaper to set up a private system and couple it to the public one, so both can be accessed from all terminals (Maurer and Sebestyen, 1983).

Such a coupled public-private system is really a two-level hierarchy. As the size of the system grows, one can imagine a deeper hierarchy developing, with the lowest level holding corporate information, the second level containing local or regional information, the third level supplying national information, with the top level dealing in international information. In this way, information providers could get the coverage they want. Local restaurants would probably want to have their menus available only in region, but major hotel chains could make their reservation systems available world wide by using the top level.

9.6. EXAMPLES OF THE APPLICATION LAYER

In this section we will examine our running examples for the last time to see how they deal with file transfer, electronic mail, virtual terminals, and so on.

9.6.1. The Application Layer in Public Networks

The OSI (and CCITT) work has produced models and standards for file transfer, electronic mail (MOTIS/MHS), virtual terminals, and other applications. We will examine the first three below. The OSI directory service and job transfer and manipulation sections in the text have already described the OSI work.

File Transfer, Access, and Management

The OSI **FTAM** (File Transfer, Access, and Management) model is based on the idea of a virtual filestore that is mapped to a real filestore by software. The OSI virtual filestore definition is highly complex, with many bells and whistles, so our discussion will only give the general flavor, not all the details. For those, please consult the standard itself, ISO 8571.

FTAM has one basic file type, the hierarchical file, but it is possible to specify constraints that give other types as special cases. One special case is the unstructured file, which is manipulated as a unit, with no filestore operations defined on subsections of the file. Another important case is the sequential file, in which the records can be accessed by their labels or positions. Files have the attributes listed in Fig. 9-5.

The virtual filestore is highly connection oriented, with a series of nested **regimes**. Each regime has a series of operations that are permitted during it. When a connection is established, using the *F-INITIALIZE* primitive, FTAM enters what is called the **FTAM association regime**, as shown in Fig. 9-25. During this regime, the FTAM user is permitted to perform operations that manage the virtual filestore.

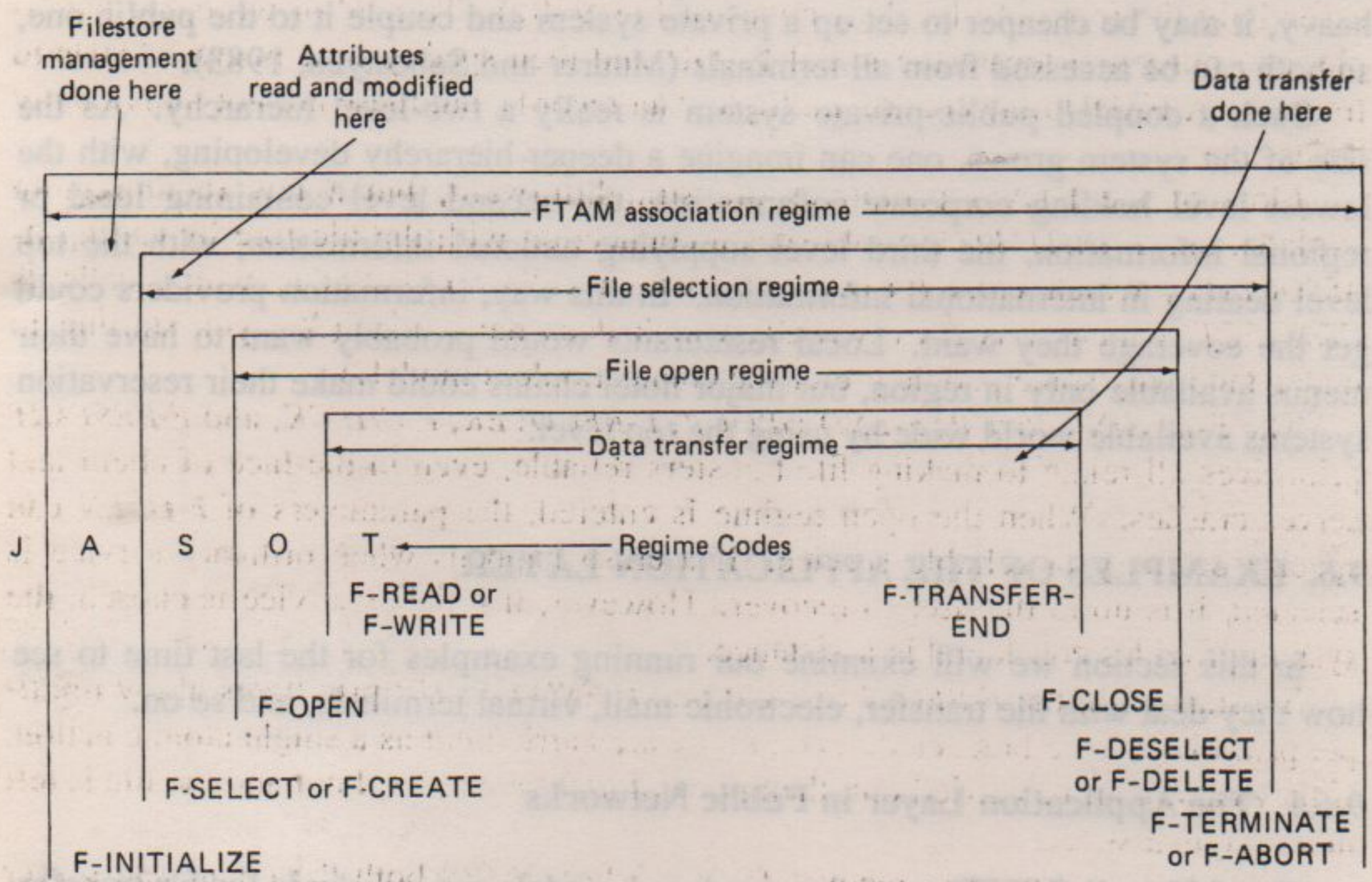


Fig. 9-25. The FTAM regimes.

To access a specific file, the user then executes an *F-SELECT* primitive to select an existing file, or an *F-CREATE* primitive to create a new one. In both cases, the user now enters the **file selection regime**, in which the file attributes can be read or modified. The next step is initiated with *F-OPEN*, which enters the **file open regime**, in which data can be transferred. Normally, the regimes are exited in the reverse order they are entered, although in emergencies it is possible to bail out and abort everything.

While the existence of *F-SELECT* and *F-OPEN* may seem redundant, with some effort it is possible to construct situations where having the two regimes is useful. Since the OSI philosophy is to include every feature that anybody might ever

conceivably need, no matter how remote the chance, they were both included. Both primitives have parameters that specify the lock needed, if any. In particular, it is possible to select a file with a shared lock (for reading), then open a file with an exclusive lock (for writing). After exiting the selection regime, the exclusive lock is released, but the shared lock is still in place, so the file can be reopened for reading without any danger of losing the shared lock. In this way, an application can go back and forth between reading and writing phases without any danger of strangers sneaking in and locking the file for writing between an *F-CLOSE* and the following *F-OPEN*.

The list of FTAM primitives is given in Fig. 9-26. Some of these relate to regime management, and others to the specific operations on files, which are listed in Fig. 9-6. A few of the primitives operate on **FADUs (File Access Data Units)**, essentially the (possibly labeled) nodes of Fig. 9-4(c). To read or write, for example, one must first locate the initial FADU and then specify how many FADUs to transfer. A FADU may contain multiple data records, but these are not individually accessible.

Most of the FTAM primitives are straightforward, but a few require more explanation. *F-INITIALIZE* establishes a connection with the filestore by calling *A-ASSOCIATE*, which calls *P-CONNECT*, which calls *S-CONNECT*, which calls *T-CONNECT*, which calls *N-CONNECT*, which does the work. (Are you starting to understand why some people prefer the connectionless model?)

The *F-BEGIN-GROUP*, *F-END-GROUP*, *F-RECOVER*, *F-CHECK*, and *F-RESTART* primitives all relate to making file transfers reliable, even in the face of client and server crashes. When the open regime is entered, the parameters of *F-OPEN* can specify ordinary or reliable service. If a crash happens when ordinary service is selected, it is up to the user to recover. However, if reliable service is chosen, the system provides considerable assistance.

To start with, the *F-BEGIN-GROUP* and *F-END-GROUP* primitives allow multiple primitives to be bracketed, so that they are carried out as a single atomic action. If they cannot all be carried out, then none of them are carried out and the file is left in its original state.

F-RECOVER is used to recover from crashes. After both the client and server machines are back in operation, the FTAM user can invoke this primitive to return to the regime that was in effect at the time of the crash, with the same parameters and same locks.

F-RESTART is used to resynchronize from minor disturbances that do not break the association, but do cause the user to lose track of the current state of the transfer.

F-CHECK is used to establish checkpoints. It uses the session service's minor sync points. After a checkpoint has been confirmed, any subsequent *F-RECOVER* or *F-RESTART* will fall back to the most recent checkpoint. By making occasional checkpoints during a long file transfer, crashes do not require starting the entire transfer all over from the beginning.

FTAM Service primitive	Confirmed	Regime	Description
F-INITIALIZE	X	I	Establish a connection with the filestore
F-TERMINATE	X	A	Release a connection with the filestore
F-U-ABORT		all	User-initiated abort
F-P-ABORT		all	Provider-initiated abort
F-SELECT	X	A	Select a file for manipulation or transfer
F-DESELECT	X	S	Terminate a selection
F-CREATE	X	A	Create a file
F-DELETE	X	S	Destroy a file
F-READ-ATTRIB	X	S	Read file attributes
F-CHANGE-ATTRIB	X	S	Modify file attributes
F-OPEN	X	S	Open a file for reading or writing
F-CLOSE	X	O	Close an open file
F-BEGIN-GROUP	X	A	Mark start of an atomic action
F-END-GROUP	X	A	Mark end of an atomic action
F-RECOVER	X	A	Recreate open regime after a failure
F-LOCATE	X	O	Move file pointer to a specific FADU
F-ERASE	X	O	Destroy a FADU
F-READ		T	Read one or more FADUs
F-WRITE		T	Write one or more FADUs
F-DATA		T	Data arrival (F-DATA. indication only)
F-DATA-END		T	End of FADU (F-DATA-END. indication only)
F-TRANSFER-END	X	T	End of data transfer regime
F-CANCEL	X	T	Abruptly terminate data transfer regime
F-CHECK	X	T	Set a checkpoint
F-RESTART	X	T	Go back to a previous checkpoint

Fig. 9-26. The FTAM primitives.

FTAM uses a protection scheme so general that no real file server can hope to implement all of it, but most will be able to implement at least some of it. Associated with each file is an access control attribute. This attribute is a set of arbitrarily many five-element lists. Each list contains a bit map, a concurrency key, a user identifier, a list of passwords, and a user location, as shown in Fig. 9-27.

When a user selects a file, the *F-SELECT* contains parameters telling the operations that are needed, the concurrency needed, and optionally some passwords. The user and location identifiers are already known from the *F-INITIALIZE*. Each list in the control attribute is checked in turn to see if it grants permission. Permission can be granted on the basis of user identifier, password, or location (one of these in one list is enough).

In the example of Fig. 9-27, two lists are associated with the file. The first list grants permission to read the attributes, read the data, or extend the file (see the column *bitmap* in Fig. 9-6 for the list of operations that can be specified). The

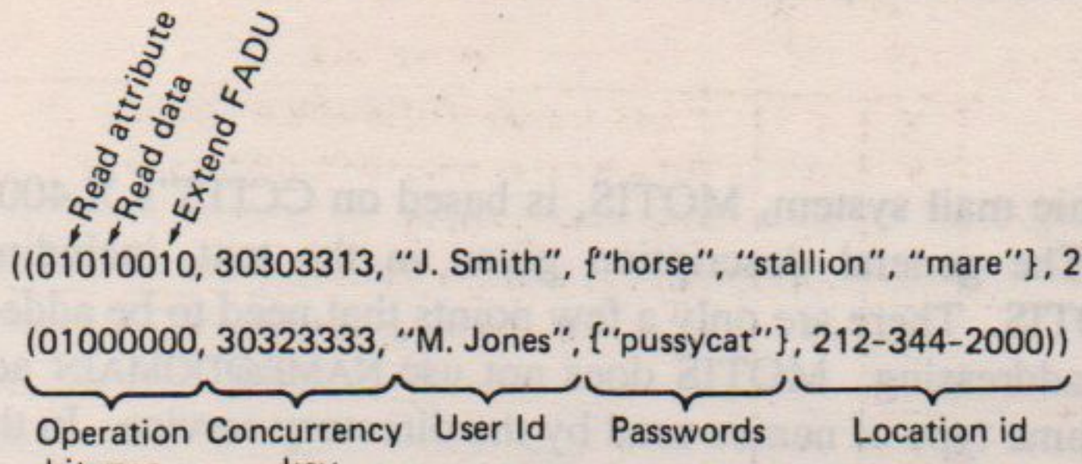


Fig. 9-27. An example access control attribute.

second list only allows reading of the attributes but not reading or writing the data or anything else.

If any of the operations requested by the *F-SELECT* are turned off in the bit map, that list fails. If all the operations are valid, the concurrency key for the list entry is checked against the type of locking the user wants for each operation. There are four possibilities per operation:

0: I want a shared lock (for reading).

1: I want an exclusive lock (for writing).

2: I will not do the operations, but others may.

3: No one may do the operation.

In Fig. 9-27, for example, J. Smith will not be granted access if he wants an exclusive lock for reading attributes because the second digit of the concurrency key is a 0 (shared lock). If the concurrency is all right, then either the user identifier, the passwords, or the location identifier must agree. It is only necessary to pass one test, so an installation could decide just to check who was making the request or where it was coming from. Alternatively, it could allow anyone quoting the proper passwords to perform the operations, without regard to who they were or where they were.

FTAM is so complex that it is unlikely that anyone will implement the whole thing. To prevent the appearance of many incompatible subsets, the FTAM standard has split the features up into 9 subsets, one of which is required and eight of which are optional. This organization reduces the number of possible combinations to 2^8. At *F-INITIALIZE* time, a check is made to see what the user and the filestore have in common. Requests falling outside the common subset cannot be used.

The FTAM protocol is very complicated, but it could hardly have been otherwise given all the features that have to be supported. Each primitive has one or more PDUs associated with it, each of which is defined in ASN.1. The standard

also defines a finite state machine telling which actions are to be done for each possible event, such execution of a primitive or arrival of a PDU.

Electronic Mail

The OSI electronic mail system, MOTIS, is based on CCITT's X.400 series of recommendations. The general description given in the text, including all the figures, apply to MOTIS. There are only a few points that need to be added.

One of these is addressing. MOTIS does not use NAME@DOMAIN addressing. Instead, it uses the same type of names used by the directory service. In the context of MOTIS, they are called **O/R** (**Originator/Recipient**) names. Thus a name is any string of attributes that allows the name to be unambiguously looked up by the directory service. The value of this system should be immediately obvious: the directory service can be consulted for PSAP addresses, routes, and any other technical information people are likely to forget.

In practice, certain additional requirements will be added to this basic scheme. For example, for international traffic, the country name must be specified even if the attributes given are unambiguous. Thus an O/R name of:

ORG = KANGAROO EXPORTS LTD, DEPT = SUPPLY, NAME = BRUCE BOOMER

would be allowed in Australia but not elsewhere, even if no other organization in the world was called Kangaroo Exports Ltd. The reason is obvious: without a country specification, the originator's message transfer agent would have to search the top-level directory of every country in the world to find the right organization. Even if by luck the search was successful quickly, the rest of the world would still have to be searched to see if there were multiple matches (and thus an ambiguous O/R name).

The detailed mechanism for sending mail works like this. The user constructs the message using a user agent (mail program) of the type described in the text (and Fig. 9-13). The user agent then passes the message to the message transfer agent using a *SUBMIT.request* primitive. The parameters of this primitive consist of the fields in the envelope, plus the message itself (header plus body).

After the message transfer agent has accepted the envelope and message, it looks up the address of the next message transfer agent along the route and hands it the whole thing using **RTS** (**Reliable Transfer Service**), which uses the *P1* protocol of Fig. 9-11.

The reliable transfer service is designed to avoid losing mail, even in the face of repeated crashes. This service has seven primitives, as shown in Fig. 9-28. The service is implemented by establishing an association between the message transfer agents, and using a stop-and-wait protocol on it. After each message is sent, it is confirmed before another one may be sent.

Reliability is achieved by using session activities. Each message is sent as a separate APDU bracketed by its own activity. If the association is broken due to a

RTS Service primitive	Confirmed	Description
RT-OPEN	X	Establish an association
RT-CLOSE	X	Release an association
RT-U-ABORT		User-initiated abort
RT-P-ABORT		Provider-initiated abort
RT-TRANSFER	X	Transfer one message
RT-TURN-PLEASE		Please give me the "token"
RT-TURN-GIVE		Here is the "token"

Fig. 9-28. The reliable transfer service primitives.

crash of either partner, the session layer recovery mechanism is used to redo the activity in an unambiguous way. For long messages, minor sync points can be used to reduce the amount of retransmission needed for recovery.

It may seem strange that MOTIS does not use FTAM for transporting messages, even though FTAM also provides for reliable transfer using session activities. The reason is that MOTIS is rooted in the 1984 X.400, which predates FTAM.

Virtual Terminals

The OSI **VTS (Virtual Terminal Service)** is very similar to the form mode virtual terminal described in the text. Both the synchronous and asynchronous modes are supported. The central idea behind the model is that of a **conceptual communication area**, which is the shared data structure. This area contains five different kinds of objects, which we will now discuss in turn.

The **conceptual data store** is a three-dimensional array of characters chosen from some agreed upon alphabet. One consequence of this standard having been written by an international committee was the demand for support of diacritical marks, something not needed in English, but required in many other languages. Symbols such as é and è are used in French, and umlauts are used in German, for example. The solution chosen by OSI was to have a special character set of non-spacing diacritical marks that may overlay regular characters. Other than superimposing one of these on a regular character, overstriking is not permitted.

Associated with each character position are attributes for the font, emphasis, foreground color, and background color. The choice of character sets and the allowed values for the attributes are determined when the virtual terminal association (connection) is established.

The second kind of object in the conceptual communication area is the **control object**. Control objects can be used for handling terminal functions that are not related to displaying text, such as ringing bells, turning lights on and off, and modeling function keys and mouse buttons.

The standard does not specify the semantics of control objects in general, although it does provide one example—echo control. If an asynchronous mode virtual terminal were set up to collect entire lines before forwarding them, no characters would appear on the screen until after the entire line had been typed and transmitted and a response received. To get around this problem, there is a control object that can be set to enable local echoing, so that characters appear on the screen as they are typed. However, the computer can disable echoing when passwords have to be typed, by setting the echo control object to FALSE.

The third kind of object in the conceptual communication area is the **device object**. There is one device object for each real device attached to the terminal (e.g., keyboard, display, printer). It handles the interface with the real device, making whatever conversions are necessary.

The fourth kind of object in the conceptual communication area deals with token ownership when the virtual terminal is being operated in synchronous mode. It keeps track of who has the token, and is concerned with token passing.

The fifth kind of object holds the parameters and ASN.1 definitions used by the virtual terminal service and protocol. When a virtual terminal association is established, the two parties have to agree on a large number of parameters and options. One side can propose a value and then the other side can accept or reject it. This negotiation process continues until all the values have been settled. Alternatively, the two sides can agree to use a standard profile, providing a complete parametrized description of the virtual terminal properties. Some of the OSI virtual terminal parameters are listed in Fig. 9-29.

Mode: synchronous or asynchronous
Number of dimensions: 1, 2, or 3
Is the terminal capable of erasing (CRTs) or not (hardcopy)
Character sets allowed
Fonts allowed
Emphasis types allowed
Foreground colors allowed
Background colors allowed
Maximum x coordinate (or infinite)
Maximum y coordinate (or infinite)
Maximum z coordinate (or infinite)
Addressing constraints

Fig. 9-29. Some of the OSI virtual terminal parameters.

The mode, number of dimensions, erasure capability and attributes are straightforward. The maximum bounds on the (up to) three dimensions are also what you would expect. Infinite bounds are allowed, in which case the scrolling mechanism with the update window scheme described earlier in the chapter is used. In addition, several parameters are present to allow addressing constraints to be described.

Some terminals may not be able to backspace, for example, and simple hard copy terminals may not permit any random access at all.

The virtual terminal can perform all the operations listed in Fig. 9-19, as well as several others related to cursor motion within the update window.

It also supports three kinds of delivery control: none, simple, and quarantine. When no delivery control is used, the virtual terminal software determines when to send PDUs. With simple delivery control, the same rule applies, except that a primitive *VT-DELIVER* is provided to flush the buffer. Finally, with quarantine deliver, PDUs are only sent when *VT-DELIVER* is invoked.

The basic VTS primitives are listed in Fig. 9-30. The first four primitives are the same as in FTAM and many other applications. The next one allows the peers to negotiate the use of a standard profile, instead of negotiating the individual parameters one by one. If no standard profile is available, the six primitives with *NEG* in their names can be used to negotiate the parameters one by one.

VTS Service primitive	Confirmed	Description
VT-ASSOCIATE	X	Establish a connection
VT-RELEASE	X	Release a connection
VT-U-ABORT		User-initiated abort
VT-P-ABORT		Provider-initiated abort
VT-SWITCH-PROFILE	X	Negotiate the use of a standard profile
VT-START-NEG	X	Start the parameter negotiation phase
VT-END-NEG	X	End the parameter negotiation phase
VT-NEG-INVITE		Invite the peer to propose a parameter value
VT-NEG-OFFER		Propose a parameter value
VT-NEG-ACCEPT		Accept a proposed parameter value
VT-NEG-REJECT		Reject a proposed parameter value
VT-DATA		Data operation
VT-DELIVER		Flush out all buffered data
VT-ACK-RECEIPT		Acknowledge VT-DELIVER
VT-GIVE-TOKEN		Pass the token in synchronous mode
VT-REQUEST-TOKEN		Request the token in synchronous mode

Fig. 9-30. The basic VTS primitives.

The *VT-DATA* primitive is used for all the operations listed in Fig. 9-19. A parameter is used to specify the operation. There does not appear to be any overriding reason to lump all the data operations in one primitive. The committee could equally well have decided to have a single primitive *NEGOTIATE* with a parameter telling which of the six possibilities was desired and have had each of the operations of Fig. 9-19 be a separate primitive.

In fact, that would have made more sense, since the most common PDU is the

one corresponding to *VT-DATA*, and this PDU will need a field giving its type, and then a second field giving the operation. If each operation were a separate primitive, corresponding to a separate PDU, these important PDUs would be shorter.

The last primitives deal with delivery control and token management. We have already seen *VT-DELIVER*. *VT-ACK-RECEIPT* is used to acknowledge it.

VTS also has facilities for handling forms. A form is constructed as a sequence of fields, each field built up from one or more rectangles. The composite field need not be rectangular. Each field is addressed by its field number, independent of its position in the character array.

Field addressing is useful for making application programs independent of the screen geometry. Consider, for example, an application that uses many fields. When run on a 66×80 terminal the position of the fields on the screen will be different than on a 25×132 terminal. By allowing the program to address fields by number instead of by position, the program does not have to be aware of the layout of the fields on the screen. At the time the association is established, the terminal downloads the form layout from the library for its terminal type, and thereafter screen coordinates need not be used.

Commands are available to create fields, delete fields, display text in fields, and erase text from fields. Furthermore, restrictions can be imposed on fields, such as requiring that the input entered into a field consist of exactly four decimal digits. Fields can also be made read only to prevent operator input.

Directory Service

The directory service model described in the text is the one ISO has chosen. The details of this service are being carefully worked out with CCITT, since it is the telephone companies and PTTs that have traditionally provided telephone books and operator-supplied information.

To prevent different names from being used for the same attribute in different countries and organizations, work is in progress to standardize a certain number of common attribute names and the forms of their values. It is also likely that all queries will be required to include certain attributes, such as the country and administrative domain, although certain defaults will be provided if they are not listed.

Job Transfer and Management

OSI JTM also closely follows the model described earlier in the chapter. Rather than talking about files, OSI uses the term **document** as a kind of generalized information carrier. A document may contain several files. The JTM service deals with the manipulation of documents.

The JTM software performs its work by interacting with four kinds of agencies:

1. **Initiation agency**: the process (or person) that submits the work.
2. **Source agencies**: file systems that provide needed files.
3. **Sink agencies**: file systems that accept files for storage.
4. **Execution agencies**: machines that run jobs.

JTM's main function is to issue primitives to these agencies giving them work to do and to accept primitives from them containing information and work for JTM itself. The OSI JTM primitives are listed in Fig. 9-31. They fall cleanly into four groups.

The first group consists of *J-INITIATE* (which has several variants), and is used to pass work specifications to JTM for processing. Each work specification tells in detail what is to be done.

JTM Service primitive	Issued by: Initiator	JTM provider	Execution agency	Sink agency	Source agency	Description
J-INITIATE	X					Create and initiate a work specification
J-GIVE		X				Ask source agency to give JTM a document
J-DISPOSE		X				Pass a document to execution or sink agency
J-ENQUIRE		X				Ask source or execution agency for document names
J-SPAWN			X			Create a subjob
J-MESSAGE			X			Report on work in progress
J-END-SIGNAL			X	X		Report completion of JTM work
J-STATUS		X				Ask about status of work
J-HOLD		X				Temporarily suspend work
J-RELEASE		X				Continue suspended work
J-KILL		X				Abort work abruptly
J-STOP		X				End work gracefully

Fig. 9-31. The JTM primitives.

The second group contains three primitives issued by the JTM provider to ask for documents, deposit documents, and inquire about document availability. All of them are confirmed, with the agency addressed accepting or rejecting the work request, or providing (or not providing) the requested information.

The third group consists of three primitives used by execution and sink agencies to create subjobs, give status reports, and announce completion of work.

The fourth and last group of primitives is used by the JTM provider to control the behavior of previously issued work requests. JTM can ask about the status of the work, suspend and resume it, and kill it. Two ways are provided for terminating work, one abrupt, which just kills it immediately without regard to the consequences, and one that makes an attempt to kill it gracefully, preferably with a report on how far things were and so on. On *J-KILL*, execution agencies are expected to roll back any permanent effects of actions already taken, to the extent possible (using CCR primitives), but on *J-STOP* permanent actions are committed (using CCR primitives).

9.6.2. The Application Layer in the ARPANET

The ARPANET has standard protocols for file transfer, electronic mail, virtual terminals, and many other applications. They differ from their OSI counterparts in a fundamental way: they were all designed bottom up. In the beginning, the protocols were simple, with few features. As it became obvious that new features were needed, they were added. The result of this process is a set of protocols that actually handle the situations that people need.

In contrast, the OSI designers often had little practical experience to guide them and tended to include every feature that anyone thought might conceivably be of some use. On the other hand, for every OSI service, there is a semiformal service specification telling what the service does. The ARPANET lacks these service specifications.

File Transfer

The ARPANET file transfer model is not based on the idea of a virtual filestore. Instead, it is based on the idea of transferring a file from one real machine to another, taking into account differences between the machines. These differences may require some conversions to be performed during the transfer, something the protocol can handle. For example, transferring a file for subsequent retrieval, bit-for-bit, is different than transferring a file to a printer that requires certain carriage control conventions.

Many of the original ARPANET hosts were 36-bit machines, including DEC-10s and DEC-20s, as well as MULTICS machines. Other machines, such as IBM mainframes, had 32-bit words. Consequently the problem of transferring data between machines with different word sizes was taken seriously. OSI essentially ignores the problem.

As a simple example of the problems that can occur, consider transferring a file from a 32-bit machine to a 36-bit machine (the easy case—the other way is harder). At least four reasonable models exist, as shown in Fig. 9-32. In Fig. 9-32(a) three 32-bit words are shown. Each contains four characters. In Fig. 9-32(b) and (c),

each 32-bit word is placed in a 36-bit word, left and right adjusted, respectively, with 4 zero bits. In Fig. 9-32(d) each 8-bit character is placed in a 9-bit character, with the high-order bit zero. In Fig. 9-32(e) the bits are packed tight, with no wasted space. Nine characters are packed into two words.

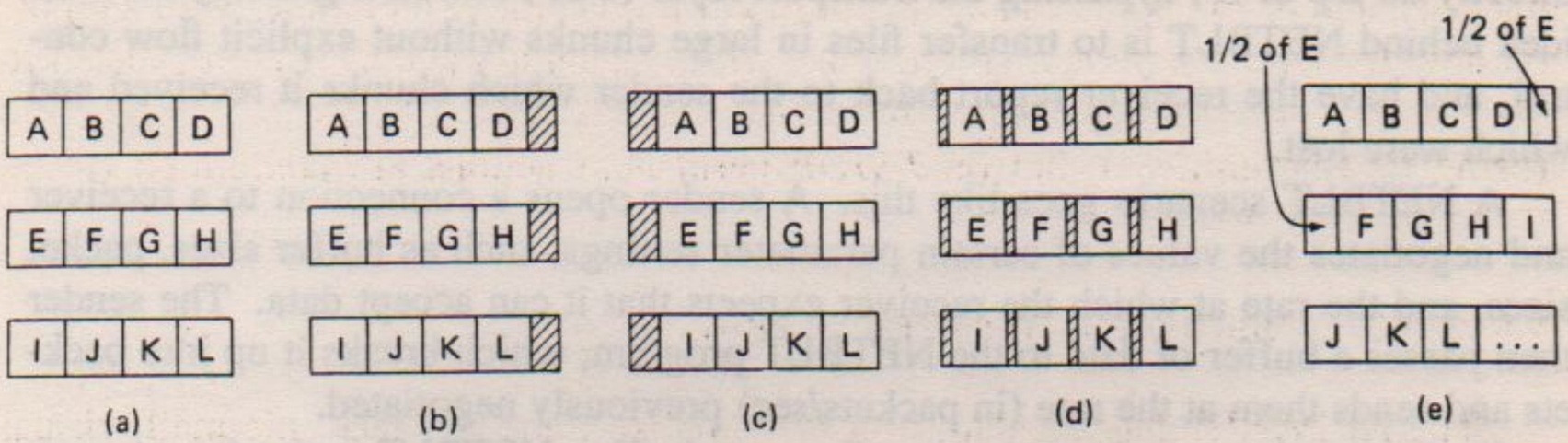


Fig. 9-32. (a) 32-bit words. (b) - (e) Different ways of storing the bits in 36-bit words.

The file transfer protocol, **FTP**, distinguishes four file types: image, ASCII, EBCDIC, and logical byte files. Image files are transferred bit for bit, without change. ASCII files are the standard for text exchange, except between IBM mainframes, in which case EBCDIC is used. Logical byte files are binary files with a byte size different from 8 bits.

FTP recognizes certain types of structure, roughly analogous to the OSI unstructured, sequential, and hierarchical files. There are unstructured files, files that are sequences of records, and random access files that are collections of pages. An unstructured file can be of image, ASCII, EBCDIC, or logical byte type.

A record-oriented file consists of a sequence of fixed size records. Many mainframes expect all input to be 80-character card images and all output to be 132-character print files. Many of these files contain a large number of space characters.

Paged files consist of blocks of data, each with a header giving its size, position and type. One place where these files are useful is for files with holes in them. In UNIX, a process can create a file and write 1 byte into it, then seek to byte 1 billion and write a second byte. On the disk, only two blocks are used, but most file transfer protocols would send the file as 1,000,000,001 bytes. With paged files this can be avoided. Paged files are also used to indicate that some pages may contain bad data due to magnetic tape read errors or measuring instrument problems.

File transfer can occur in three different modes. Stream mode is used for ordinary files. Compressed mode is used to squeeze out runs of consecutive characters, typically spaces, in record-oriented files. Block mode is used for paged files, in which each page has its own header. Checkpointing and error recovery is provided for compressed and block mode files.

A wide variety of commands are supported by FTP. Some of them have to do with sending and receiving files, others manipulate directories, and others relate to

setting parameters and modes for transfers. All in all, the model is quite different from the OSI virtual filestore model.

In addition to FTP, a second file transfer protocol, **NETBLT (NETwork BLock Transfer)** is used for very fast file transfers (Clark et al., 1987). It runs directly on top of IP, bypassing the transport layer (TCP) and all higher layers. The idea behind NETBLT is to transfer files in large chunks without explicit flow control, and have the receiver report back to the sender which chunks it received and which were lost.

A NETBLT scenario goes like this. A sender opens a connection to a receiver and negotiates the values of certain parameter settings, such as buffer sizes, packet sizes, and the rate at which the receiver expects that it can accept data. The sender then passes a buffer of data to the NETBLT program, which breaks it up into packets and sends them at the rate (in packets/sec) previously negotiated.

In the simplest version, after sending one buffer, NETBLT waits for a report about which packets arrived safely and which were lost or damaged. The packets that did not make it are then sent again. At this point the sender is informed that the buffer has been sent, and allowed to offer a new one. This algorithm is repeated until the entire file has been transmitted.

An improvement to the basic algorithm is not to use stop-and-wait per buffer, but to allow the sender to supply several buffers at once. In this manner, NETBLT can start sending the second buffer, even before the first one has been acknowledged.

Unlike nearly all other protocols we have studied, which use a flow control algorithm permitting a sender to send a certain number of packets before stopping, the NETBLT protocol uses a rate-based flow control. A sender can transmit packets whenever it wants to, but it must not send more than a certain number of packets per second. If the rate negotiated initially proves unsatisfactory, it may be renegotiated up or down during the transfer. Experience has shown that this protocol can achieve extremely high data rates on networks ranging from LANs to satellite channels due to its low overhead.

Electronic Mail

Electronic mail was essentially pioneered by the ARPANET and is still its primary use. The mail format is defined in a document called **RFC 822**, the only name it has. It has evolved over the years from earlier mail standards. The protocol is called **SMTP (Simple Mail Transfer Protocol)**.

RFC 822 has been designed for sending messages containing lines of ASCII text. No other character sets are supported. Neither are mechanisms for sending facsimile, digitized voice, pictures, or other forms of communication, although other ARPANET standards do deal with some of these.

RFC 822 does not make any distinction between the envelope and the message. Instead, each piece of mail is simply a file containing certain header fields at the

top. Each header field is an ASCII keyword, followed by a colon, followed by the value. For example, consider a typical line identifying the sender:

From: TOM@PHYSICS.HARVARD.EDU (Tom Jones)

Any string in parentheses is treated as a comment and is ignored. There are some specific rules about how long lines are treated, the significance of white space (spaces and tabs), and other layout issues, but we will not go into them here.

One contrast between RFC 822 and MOTIS is worth mentioning. In RFC 822, the headers are all in ASCII and are part of the message (in-band signaling), making it easy for people to change them with their editors. While this feature is often convenient, it also provides unwary users with enough rope to hang themselves. MOTIS decided to avoid this problem by clearly separating the headers and the content, and furthermore encoding all the headers in binary (by defining an ASN.1 type for the envelope) to make it much harder for users to casually tamper with them.

The ARPANET (and ARPA Internet) use NAME@DOMAIN addressing. Among the top-level domains are EDU (educational institutions), COM (companies), GOV (government), MIL (military sites), ORG (nonprofit organizations), and the two- and three-character abbreviations for countries other than the U.S. Domains can have subdomains, as we have pointed out before. All addresses in header fields have this format.

The RFC 822 headers shown in Fig. 9-33 are divided into four rough groupings: envelope functions, address headers, message contents, and miscellaneous. The *Sender* and *To* fields are the originator's and recipient's network addresses.

As messages move around the network, *Receive* header lines are added at each hop. These lines list the immediate neighbor that passed the message, the identity of the receiving machine, the physical medium on which the message arrived (e.g., the ARPANET or some LAN), and the protocol that was used for delivery. They also give the date and time. As a result, when a message reaches its destination, it contains enough information that its exact path can be reconstructed to monitor network operation.

The remaining fields are straightforward. With the exception of *Keywords*, all of them have a reasonable counterpart in MOTIS. In addition to the ones we have shown, about a half-dozen other fields exist. These mostly relate to forwarded mail (who forwarded it, from where, to whom, when, and so on).

One field that is worth mentioning is *Encryption*. Because RFC 822 does not make any distinction between envelopes and messages, all the header fields are in plaintext. Only the body is encrypted. An intruder is free to read the *Subject*, *Keywords*, and other header lines, which may provide valuable information. In MOTIS, encryption is handled by the envelope, so the entire message, including the headers, is encrypted. Only the envelope parameters are in plaintext, and none of these relate to the content of the message.

With RFC 822 so widely used and MOTIS so up-and-coming, there is a serious

Field	Description
Sender	Address of the person who sent the message
To	Recipient's address
Received from	Where did message come from
Received by	Who received the message
Received via	On which physical medium did it arrive
Received with	Which protocol did it use
From	Name of the person who sent the message
Reply-To	Address to reply to
Cc	Addresses to which copies are to be sent
Bcc	Addresses of blind copies
In-Reply-To	Message-ID being replied to
References	Other messages cited
Subject	What the message is about
Keywords	Content descriptors
Date	When was the message sent
Message-ID	Message identification
Comments	User defined
Encrypted	Index into encryption key table

Fig. 9-33. RFC 822 header fields. The fields concerned with forwarded mail are not listed.

compatibility problem. One solution is **mail gateways**, which accept messages in one format and forward them in the other. Conversion from RFC 822 to MOTIS is relatively straightforward. The *Keywords* field is just dropped and the rest can be mapped onto some MOTIS field.

The hardest part of the mapping is the addressing since the RFC 822 NAME@DOMAIN addresses are radically different from the attribute-based O/R addressing used by MOTIS. If the *domain* field is structured hierarchically, as in Tom Jones' address above, an implicit mapping is

C = USA, ORG = HARVARD, DEPT = PHYSICS, NAME = TOM

The problem mapping between RFC 822 and X.400 (MOTIS) addresses is studied in more detail by Henken (1987).

Going from MOTIS to RFC 822 is much harder, since many of the MOTIS fields do not exist in RFC 822. Those fields that are irrelevant (e.g., everything about conversions) can just be dropped. For those fields that might be important, such as digital signatures, RFC 822 provides an escape hatch. Header fields starting with *X-* are accepted by the mail software as syntactically valid headers, but are not examined or acted upon. Thus the mail gateway could create special fields such as *X-Sender-Signature* to encode these MOTIS fields. As time goes on, some of these will probably become "legal" and drop the *X-* prefix. As an aside, frivolous headers (e.g., X-Fruit-Of-The-Day: Kumquat) are not totally unknown on the net.

Most mail is transported using the SMTP protocol. To use it, a host initiates a TCP connection to another host, which may either be the ultimate destination or an intermediate stop along the way. Once the connection has been set up, the initiator sends a *MAIL* command, identifying the sender, primarily so the destination knows where to send error messages, should they be required. If the receiver is prepared to accept the mail, it sends a positive acknowledgement.

Next, the initiator sends a *RCPT* command identifying the recipient of the mail. Again, the receiver can accept or reject it. If the recipient is known and acceptable, the sender issues a *DATA* command, followed by the message as a series of ASCII lines and terminated by a line containing only a period. Character stuffing is used to allow such lines to appear within messages. Other commands are available for verifying addresses, expanding mailing lists, switching roles, and terminating connections.

Virtual Terminals

The ARPANET virtual terminal protocol, known as **TELNET**, was designed with scroll mode rather than page or form mode, terminals in mind, although some page mode terminal properties can be negotiated. The default network virtual terminal has a single line of unlimited length.

The protocol is concerned with setting up and manipulating two simplex data streams, one in each direction. It has no concept of a data structure that must be kept identical at both ends by the protocol. As keys are hit on the terminal, a stream of 8-bit bytes is transmitted over the line. The terminal process must convert whatever character code the terminal outputs to the network standard, ASCII. In addition to the 95 ASCII graphics, the CR, LF, BEL, BS, HT, VT, and FF control characters are also legal.

Furthermore, there also exists a group of 15 commands, which may be freely interspersed with the ordinary data. The list of commands is shown in Fig. 9-34. Commands are preceded by the IAC character (Interpret As Command, code 255), which says that the next character is a command. If the command is longer than one character (i.e., if it has parameters), codes 250 and 240 are used like **begin** and **end** in Pascal, to delimit the command.

Code 244 is used to interrupt a process. *Break* (243) is used to mean whatever the remote machine normally uses it for, not necessarily kill process. *Mark* is used to indicate to the terminal process where it should resume output, as above. *Abort output* is only meaningful if the remote machine has a way to terminate the output from the running process without killing it. Similarly, *Are you there* is provided by some operating systems to allow the user to confirm that although the machine may be slow, it is still there.

Codes 247 and 248 are important because they allow the user to use the local host's erase conventions on the remote machine. The clear implication of these

Code			
240	End of command	248	Erase line
241	No operation	249	Go ahead
242	Mark	250	Start of command
243	Break	251	Will
244	Interrupt process	252	Won't
245	Abort output	253	Do
246	Are you there?	254	Don't
247	Erase character	255	Interpret as command

Fig. 9-34. The ARPANET TELNET protocol commands.

codes is that when the user wants to erase a character or a line, the terminal process merely converts the local erase key into this network standard and transmits it.

The *Go ahead* command is used with synchronous terminals, to authorize the other side to send. In OSI terms, it passes the token.

Like our first example, TELNET provides for option negotiation to change the defaults and set parameters on the virtual terminal. Option negotiation may be initiated at any time, by either side, using *WILL*, *WON'T*, *DO*, and *DON'T*.

As an example of how these commands are used, a computer capable of handling full-duplex (free running) terminals, might want to find out if the terminal could also work in full-duplex mode. To do so it would send a *WILL* command, specifying its willingness to use full-duplex. The terminal would then reply with either *DO* or *DON'T*. Negotiation can also take the form of one side sending a *DO* or *DON'T* to the other one, requesting some option, with the answer being either *WILL* or *WON'T*. About two dozen options have been defined, including binary transmission, remote echoing, line and page length, and the meaning of tabs.

9.6.3. The Application Layer in MAP and TOP

MAP and TOP implement a subset of the OSI layer 7 applications. ACSE, FTAM, and directory service are fully supported. CCR and JTM are not supported. VTS is fully supported in TOP and partially supported in MAP (because virtual terminals are more important in office applications than on the factory floor). TOP also differs from MAP in another way: TOP supports X.400 whereas MAP supports a completely different kind of messaging system called **MMS** (**Manufacturing Message Standard**). MMS is intended for machine-to-machine communication, such as computer-to-robot, whereas X.400 is intended for human-to-human messages.

Nevertheless, MAP and TOP have different goals than OSI, and this leads to another difference between MAP and TOP on the one hand and OSI on the other that shows up for the first time in the application layer. The idea behind OSI is to make it possible for computers from different manufacturers to communicate with

one another. This leads to an emphasis on protocols. Service definitions are also provided, but all of them are in the form of abstract primitives as shown in Fig. 9-35.

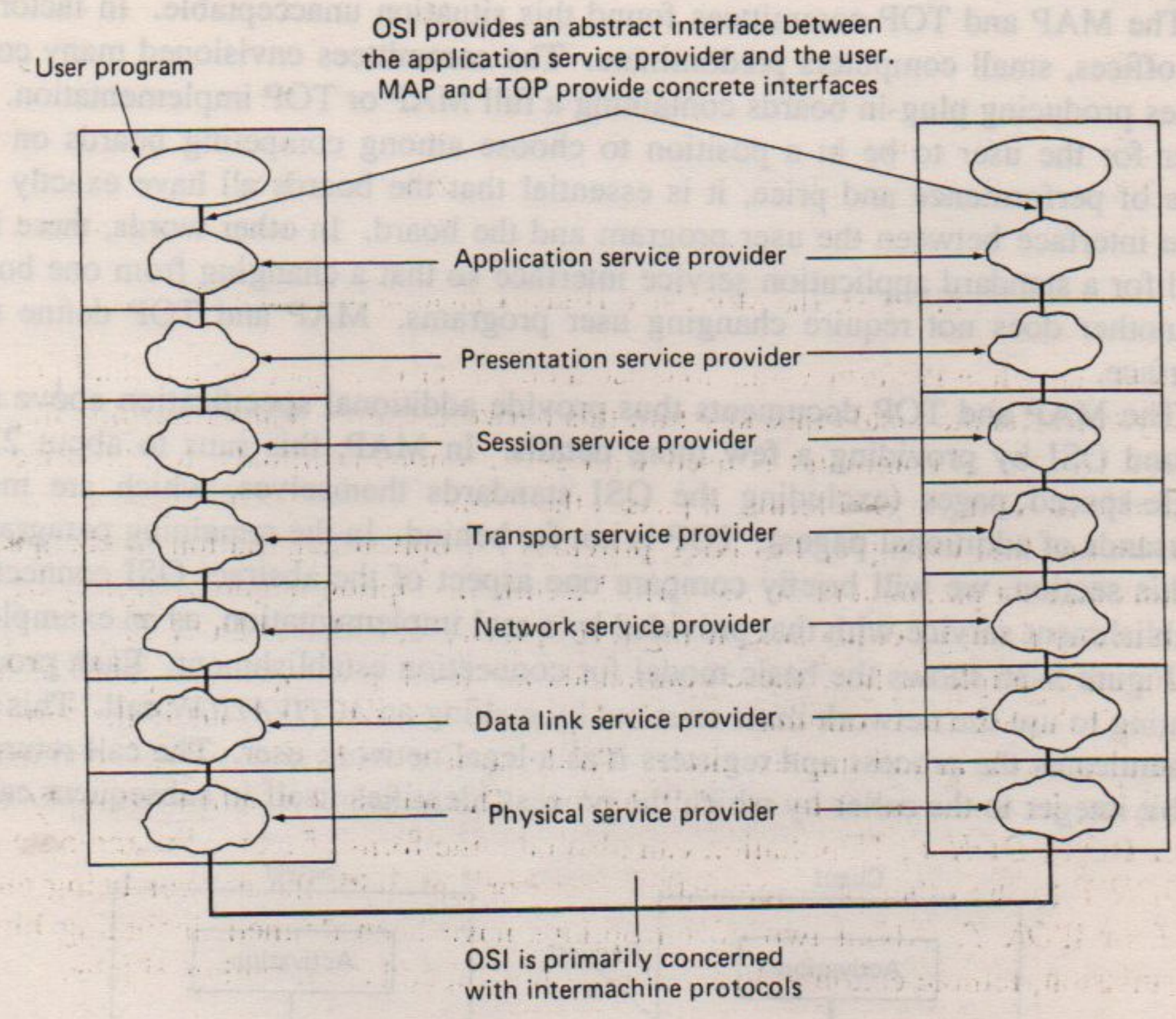


Fig. 9-35. Abstract versus concrete user interfaces.

For the intermediate layers, these abstract service definitions do not cause problems unless a user foolishly decides to acquire each layer from a different manufacturer. For a system produced by a single company, the user does not really care what the interfaces among the lower six layers are. All that matters is how programs talk to the application layer. However, the OSI standards do not specify this top-level interface in much detail, and the specifications that are given (e.g., *indication)* are not even implementable.

As a consequence of this incomplete specification, each manufacturer has necessarily defined its own interface between user programs and the application service provider. A program written using *A*'s interface will not run on *B*'s machine.

In the mainframe world, this situation does not cause problems because computer centers are like newly hatched ducklings: whatever they see first, they follow around for the rest of their lives. In ducklings it is called imprinting; in computer

centers it is called compatibility. If a computer center standardizes on brand *X* computers, the users write their programs using brand *X*'s application service interface and are not bothered by the fact that their programs will not run on brand *Y*'s computers. They do not have any of these, after all.

The MAP and TOP committees found this situation unacceptable. In factories and offices, small computers predominate. The committees envisioned many companies producing plug-in boards containing a full MAP or TOP implementation. In order for the user to be in a position to choose among competing boards on the basis of performance and price, it is essential that the boards all have exactly the same interface between the user program and the board. In other words, there is a need for a standard application service interface so that a changing from one board to another does not require changing user programs. MAP and TOP define this interface.

The MAP and TOP documents thus provide additional specification above and beyond OSI by providing a few more details. In MAP, this runs to about 2300 single-spaced pages (excluding the OSI standards themselves, which are many thousands of additional pages). TOP is not far behind. In the remaining paragraphs of this section, we will briefly compare one aspect of the abstract OSI connection establishment service with that provided by a real implementation, as an example.

Figure 9-36 shows the basic model for connection establishment. Each process wanting to use the network must start out by making an *ACTIVATION* call. This call authenticates the process and registers it as a legal network user. The call returns a 32-bit integer to the caller by which the process identifies itself in subsequent calls.

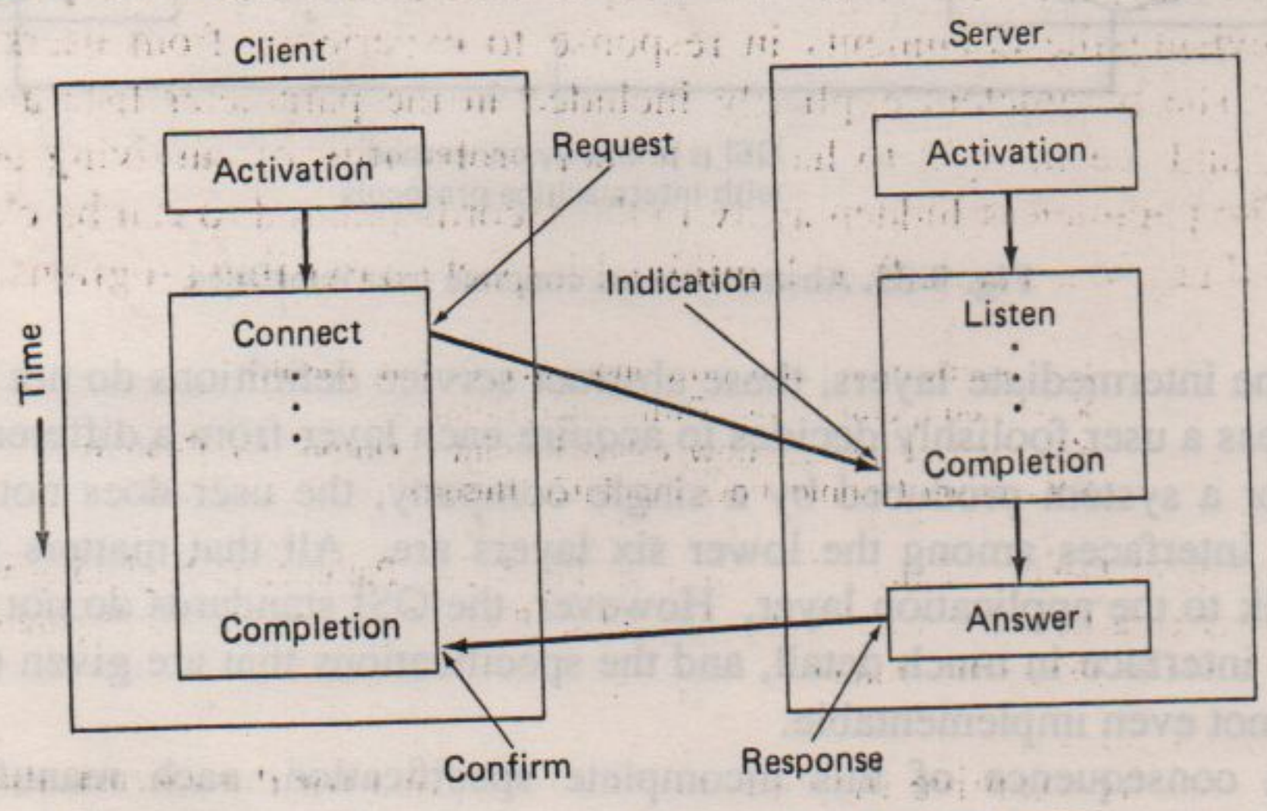


Fig. 9-36. Connection establishment in MAP and TOP.

Next, the server does a *LISTEN*. Both blocking and nonblocking versions are available. If the blocking option is chosen, the server is suspended until some client attempts a connection. If the nonblocking option is chosen, the server can compute for a while and then block later using the *WAIT* call.

When some client does a *CONNECT* call (effectively *A-ASSOCIATE.request*) the server is awakened and given the parameters contained in the incoming PDU. The awakening of the server is the *A-ASSOCIATE.indication*.

After inspecting the parameters, the server accepts or rejects the association by making an *ANSWER* call. This action is the *A-ASSOCIATE.response*. Finally, the PDU sent by *ANSWER* arrives at the client, completing the *CONNECT* (the *A-ASSOCIATE.confirm)* and establishing or denying the connection.

The similarity of this model with our detailed example of the transport layer is not entirely a coincidence. There is really no other sane way to do it. *Request* and *response* can easily be implemented by calls to library routines, but *indication* and *confirm* are fundamentally different, and cannot be implemented the same way.

MAP and TOP not only provide a model that is conceptually implementable, but they also define library procedures that can be called by user programs. By requiring MAP and TOP products to support these calls, users can ensure that their software will work without modification. The details of how the library procedures transmit their parameters to the boards (i.e., which bits go in which device registers) is not standardized because it does not affect program portability.

The parameters of the calls shown in Fig. 9-36 are given in Fig. 9-37. Some of the parameters contain explicit values passed by the caller to the application provider. Other parameters are pointers to records containing even more parameters. Result parameters are also divided this way.

The idea behind this split is that unlike ISO, which hopes to have stable standards some day, the MAP and TOP people have abandoned all hope and are constantly revising the documents in response to experience from users and implementers. The parameters explicitly included in the parameter lists are said to be **exposed**, and are thought to have a higher probability of surviving new versions intact. The parameters hidden away in the records pointed to can be changed more easily (and new ones added) with minimal impact on existing programs.

```
XX-aeactivation (who, how, inpointer, Permit, Result)
XX-listen (permit, how, Outpointer, Connid)
XX-connect (permit, how, callee, inpointer, Outpointer, Connid)
XX-answer (connid, how, inpointer, Result)
```

Fig. 9-37. Details of the calls shown in Fig. 9-36. Parameters beginning with a lower case letter are filled in by the caller. The other ones are result parameters filled in by the application provider.

Let us briefly examine the calls one by one. Just for the record, the parameters listed in Fig. 9-37 are not the official MAP/TOP names, which are somewhat obscure. The *xx_aeactivation* procedure activates an application entity (process). The *xx_* is a dummy prefix that is different for each application. For example, for FTAM, the activation call is *ft_aeactivation*.

The *who* parameter identifies the caller to the application provider (e.g., a plug-in board). The *how* parameter specifies if the call is blocking or nonblocking, not

only in this call, but in all the rest as well. The *inpointer* parameter points to a record containing the authentication information.

The remaining two parameters are filled in by the application service provider upon completion of the call. *Permit* is a random 32-bit integer used to identify the caller in subsequent calls, and *Result* is a status code that tells if the call succeeded or not. It could fail, for example, if the board can only handle a limited number of processes and it is already at the limit.

The *xx_listen* procedure contains the 32-bit identifier returned by *xx_aeactivation*, and the blocking/nonblocking flag as input parameters. The output parameters are a pointer to a record and the connection identifier. The record contains information taken from the incoming PDU, including the caller's presentation address, the proposed application context name, and a status code indicating success or failure.

The *xx_connect* procedure is used to establish an association. The remote entity can be specified either by giving its full presentation address or its entity name, in which case the the presentation address must be looked up in a directory. If the call is to a specific application entity name (e.g., "Fileserver3"), the *callee* field is used to specify it. The *inpointer* and *Outpointer* parameters point to records with parameters and results, respectively. If the connection is established, *Connid* is filled in with a 32-bit identifier for that connection.

The parameter record pointed to by *inpointer* contains the presentation address (which is null if the *callee* field is nonnull). It also contains two parameters telling how often to retry the connection if the first time fails due to a transient cause such as network congestion, and how long to wait between retries.

The record also contains a timer that gives the maximum time to wait for local resources, such as a table slot on the board. If, for example, the board can only handle *n* connections at once, and all are in use, the process calling *xx_connect* could specify that it was prepared to wait up to one minute for a connection slot.

The next parameter in the record is the application context (library of ASN.1 definitions) that the caller is proposing to use. This name is made available to the server by a field in the record returned to *xx_listen*, as we mentioned above.

Finally, there is a special application-dependent field, a pointer to yet another record. This pointer is a hook for parameters that apply only to, say, FTAM, but not to VTS.

When the server is awakened from *LISTEN*, it accepts or rejects the connection by calling the procedure *xx_answer*. The *connid* parameter must be the same one returned by *xx_listen*, to identify which call is being answered. *Result* gives the status of the call. Again here, the call could fail due to lack of table space, a parameter error, or other causes.

The record pointed to by *inpointer* provides the server's name (something the caller may not know if the connection was made to a presentation address), and the server's proposed application context. The use of a context in *xx_connect* and one in *xx_answer* constitutes a primitive option negotiation using only a tiny fraction of

the possibilities provided by OSI. If the two contexts agree, then that context is established and is used for the duration of the connection.

If they do not agree, it is anyone's guess what happens next. Presumably if the client likes the server's proposal, the connection can go on using it. Otherwise, the client will have to release the connection. This example shows how the enormous wealth of unbearably complicated options provided by OSI are often reduced to the simplest possible case in practice.

9.6.4. The Application Layer in USENET

USENET offers two services, mail and news. USENET mail has been designed to be compatible with the ARPANET mail system described above, so let us concentrate on news. News is effectively a worldwide electronic bulletin board, with hundreds of newsgroups on a great variety of subjects, both technical and other. The total traffic is many megabytes a day. Subscribers to any newsgroup can read all the messages **posted** to (i.e., deposited in) that group, and can post messages of their own. Every message posted to a newsgroup is delivered to all of its subscribers, frequently all over the earth.

The newsgroups are divided into eight broad categories based on their subject:

1. Comp: computers and their software.
2. Misc: Miscellaneous topics not otherwise classifiable.
3. News: The news system itself and its operation.
4. Rec: Recreational activities and hobbies.
5. Soc: Social issues and socializing.
6. Sci: Physics, chemistry, biology, and the other sciences.
7. Talk: Debate-oriented subjects that generate much heat and no light.
8. Alt: Alternative groups that do not fit into any other category.

Figure 9-38 lists a few of the groups, just to give the flavor of what exists. In many cases, the reader can infer the existence of other groups. For example, *comp.binaries.mac* is used for distributing Macintosh binary programs. Analogous groups exist for other popular computers. Similarly, *comp.sys.ibm.pc* is for general discussions of the IBM PC and its clones, but groups also exist for general discussions of virtually every other computer, large and small. Groups also exist for most popular computer languages.

Groups also exist for most areas of computer science, almost all programming languages, as well as for many sciences and hobbies, and political topics. Newsgroups also exist for discussing the operation of the network itself and for

comp.ai	Artificial intelligence discussions
comp.binaries.mac	Encoded Macintosh programs in binary
comp.databases	Database and data management issues and theory
comp.dcom.lans	Local area network hardware and software
comp.edu	Computer science education
comp.graphics	Computer graphics, art, animation, image processing
comp.lang.pascal	Discussion about Pascal
comp.newprod	Announcements of new products of interest
comp.os.minix	Discussion of Tanenbaum's MINIX system
comp.os.research	Operating systems and related areas
comp.protocols.iso	The ISO protocol stack
comp.sources.unix	Postings of complete, UNIX-oriented sources
comp.sys.ibm.pc	Discussion about IBM personal computers
misc.consumers.house	Discussion about owning and maintaining a house
misc.handicap	Items of interest for/about the handicapped
misc.forsale	Short, tasteful postings about items for sale
misc.jobs.offered	Announcements of positions available
misc.kids	Children, their behavior and activities
news.groups	Discussions and lists of newsgroups
rec.arts.poems	For the posting of poems
rec.bicycles	Bicycles, related products and laws
rec.food.cooking	Food, cooking, cookbooks, and recipes
rec.games.chess	Chess & computer chess
rec.humor.funny	Jokes that are funny
rec.photo	Hobbyists interested in photography
sci.astro	Astronomy discussions and information
sci.bio	Biology and related sciences
sci.space	Space, space programs, space related research, etc.
soc.college	College, college activities, campus life, etc.
soc.singles	Newsgroup for single people, their activities, etc.
talk.politics.misc	Political discussions and ravings of all kinds
talk.rumors	For the posting of rumors

Fig. 9-38. A few of the USENET newsgroups.

distributing new news software. There is even a group *rec.nude* for nudists, although it is not obvious to outsiders if a medium supporting only ASCII text is the appropriate vehicle in this case. Since each site pays its own communication costs, not every site accepts every group. In particular, many companies take only the technical groups and refuse most of *misc*, *rec*, *soc*, *talk*, and *alt*.

New groups come and go all the time. When someone proposes creating a new group, the proposal is first discussed in *news.groups* and then a vote is held. If enough people approve, the group is created. It is sometimes hard to tell whether USENET is a glimpse into the 21st century, or a New England town meeting gone international.

In principle, anyone can post a message to any group. However, experience has shown that some people abuse this privilege. When too large a percentage of the traffic is perceived by the readers as a waste of their time, the group can become a **moderated newsgroup**. In such a group, one person is designated as moderator, and is the only one able to post to the group. All attempts by other people to post to the group are automatically converted by the news software into ordinary e-mail messages to the moderator. The moderator then sifts the wheat from the chaff and posts those messages thought to be worthwhile. Moderation raises the quality of the group, but it does tend to reduce the spontaneity of the moderated groups. Some topics have both a moderated and an unmoderated group.

The news system consists of user agents and message transfer agents, just like mail systems. Let us look at each of these in turn.

The news user agent is a program that is invoked to read and send news. When it starts up, it looks in the user's news profile to see which groups the user subscribes to. It then hunts around in the analog of the message store to see if there is any unread news in these groups. If so, it displays a message announcing the first group with news and telling how many messages are present.

At that point, the user has a variety of commands available for reading the news, similar to the mail commands. It is possible to ask for a display of the subject lines, similar to Fig. 9-12. After seeing all the subjects, the user can choose to read a specific message, or just start reading them all, in order. He can also decide that none are very interesting, and ask to move on to the next group. The unread messages can either be left in their original state so that they will also be available next time, or they can be marked as "read" so they will not be mentioned next time.

If a message looks interesting, it can be read, and possibly saved to a file or printed, as well. Having read a particular message, the user now has several options. He can proceed automatically to the next message on the same topic or continue with the next message in the group, regardless of its subject.

However, he also has the possibility of reacting to the message by either sending a reply to the originator of the message (using e-mail), or by posting the reply to the newsgroup. In the former case, the reply only goes to the person who posted the message; in the latter case, it goes to all the subscribers. A message or reply may also be **crossposted** to more than one group. The news software takes care that a user only sees a given message once, even if he subscribes to several groups to which a message has been crossposted.

Since many replies refer to the original message, the user agent has an option of building a properly formatted reply that includes the original message, and then calling the editor to allow the user to edit this reply. Typically, this editing consists of removing the irrelevant portions and adding some text to the relevant parts. When the editing has been completed, the user agent delivers the message to the message transfer agent for delivery as mail or posting as news.

When a message or reply is posted, the poster is asked about the geographic

coverage desired. Messages can be posted to all members of the group, wherever they may be anywhere in the world, or they can be restricted to group members in the U.S., North America, Europe, or a smaller area such as a state or in some cases even a city. If someone in Berkeley wants to announce next Tuesday's colloquium speaker, it is unlikely that there will be many potential attendees in, say, Finland or Japan. On the other hand, someone distributing software might well be interested in reaching readers everywhere.

Discussions on USENET can go on for an extended period of time. For the benefit of newcomers, some newsgroups maintain an archive of old messages, to allow people to see what was said before they joined. Partridge et al. (1987) describe how such archive servers work in CSNET, but the same principles apply to USENET as well.

The message transfer agent for news is in many ways similar to the message transfer agent for RFC 822 mail (see Collyer and Spencer, 1987 for a discussion of the news software). This is not entirely surprising, since the formats of news messages and RFC 822 mail messages are structurally the same to permit news messages to be carried the same way as mail messages. Only the exact list of legal header fields differ. The news headers are described in RFC 1036 and listed in Fig. 9-39. News addressing uses the ARPANET NAME@DOMAIN format.

Field	Description
Newsgroups	List of newsgroups to which this message belongs
Subject	What is the message about
From	Who wrote the message
Date	When was the message posted
Path	Route the message has followed so far
Message-ID	Message identifier
Followup-To	Which newsgroups should this reply go to
References	Messages to which this is a followup
Summary	Subject of a followup message
Keywords	Descriptive terms
Distribution	Geographic distribution: NY, USA, Europe, world, etc.
Reply-To	Address to be used for e-mail replies
Sender	Who actually posted the message
Organization	Sender's employer
Lines	Number of lines in the message
Expires	When should the message be considered obsolete
Approved	Moderator's name (for moderated groups)
Control	Used for administering the news system itself

Fig. 9-39. The USENET news header fields. The first six are required. The rest are optional.

The *Newsgroups* field provides a list of groups to which the message has been

posted. A reader subscribing to any one of these groups will see the message (once). The *Subject*, *From*, and *Date* fields have the same function as in RFC 822.

News is propagated hop-by-hop over a complex and ever-changing topology of about 10,000 sites in several dozen countries. At each hop, the current receiver adds its name to the *Path* field. By looking at it, one can see how the message propagated from the source to the current site. Before forwarding a message, the news software checks to see if the destination is already in the *Path* field, in which case the message is not forwarded there. In addition, this information is used by the system administrators to see what paths are being used and to tune the routing algorithms.

The remaining 12 fields are optional. The *Followup-To*, *References*, and *Summary* fields are used for messages that are followups to an earlier posting. The *Followup-To* field is used when a message is initially posted to multiple groups, but the sender wants subsequent discussions to occur in one specific group. When anyone responds to a message, his user agent checks to see if the original message contained a *Followup-To* field, and if so, uses that as the *Newsgroups* field of the response.

When a message is a followup to an earlier message, the *References* field is included to hold the *Message-ID* of the message (or messages) being responded to. In this case, the *Subject* consists of the prefix *Re:* followed by the original subject. The *Summary* field allows the responder to provide additional information about the response.

The *Keywords* field allows a few keywords to be mentioned. Some sites archive (store) old messages for certain groups. New users can go to the archive and ask for a list of all messages containing specific keywords.

The remaining fields are straightforward and need little explanation, with the exception of *Control*. This field contains the protocol used between the message transfer agents for operating the news system. News messages are distributed over a wide variety of media, including telephone lines, leased lines, X.25 networks, the ARPANET, and other means. Conceptually, USENET is a directed graph, with the nodes being the machines, and the arcs being the transmission lines. Most arcs are bidirectional, but unidirectional arcs also exist. The graph for news is similar, but not identical, to the graph for mail.

Each arc is labeled by the newsgroups that are exchanged over it. For each link, there is a bilateral agreement about which side will call the other, how often that will occur, and which transport protocol will be used. Most of the links are short, but there are also long-haul paths between major organizations that form the **backbone.**

After two machines have established contact, one of them typically begins by sending the other all the news messages that are carried by that link, except for those that list the destination in the *Path* field. Then the flow reverses and messages travel the other way.

This algorithm is essentially flooding, but in practice, most sites receive all their

news from a single site, so it works reasonably well. The only time it breaks down is when the primary news feed is down and a site calls an alternative site. Unlike the primary, the alternative has no idea which messages have already been sent. Sending all the messages on its disk is not an attractive proposition. To solve this problem, the called machine can send a message whose *Control* field contains the value *Ihave*. The body contains the message identifiers of its message collection.

After the caller has had time to digest this message and check which ones it has and which ones it does not have, it sends a reply with *Control* set to *Sendme*, with the body containing the message identifiers of the messages it wants.

The exact mechanism by which a message posted on a small machine in some far corner of the earth is propagated to more than 10,000 machines worldwide can now be explained. When the message is posted, it is stored locally in one of the news system's internal directories. When contact is established with another machine, the message is propagated to that machine, which, in the course of time, sends it to all its neighbors. In this way, messages diffuse around the network the same way a drop of red dye diffuses into a glass of water (or the way an epidemic spreads in a population). On the average, most news is delivered within one or two days.

Ihave and *Sendme* are not the only control messages. Two others are *Newgroup* and *Rmgroup*. These are used to create and delete newsgroups, respectively. Only certain authorized people may send these messages. Several other control messages exist for administration and statistics gathering.

Although the flooding algorithm is used on most lines, within the backbone, a protocol called **NNTP** (**Network News Transfer Protocol**) is commonly used. It is described in RFC 977. The reason for the distinction is that most sites only have a single news feed, so there is little danger of receiving a message twice from independent sites. Backbone sites connect to multiple sites, so a more sophisticated protocol is needed to avoid wasting bandwidth.

NNTP provides about half a dozen commands that allow messages to be selectively downloaded. The *NEWSGROUPS* command allows one machine to ask another if it knows about any newsgroups created after a given date and time (specified as a parameter). In this way, the existence of new groups can be propagated.

Similarly, the *NEWNEWS* command allows one machine to ask for a list of news messages received after a a given date and time, usually the date and time of the last contact. Once the receiver has this list, it can request the messages it is missing with the *ARTICLE* command. The *LIST* and *GROUP* commands can be used to ask for numbers of the first and last messages held for all newsgroups or one specific newsgroup, respectively. New messages can be posted to the network with the *POST* command.

No discussion of news would be complete without some mention of the security problems it raises. Allowing foreign machines to call up and login to send or receive mail and news is a potential security threat. The protocols have to be

defined carefully to limit what remote users may do. If all that a remote user can do is deposit files in, or read files from, a specific directory, there is relatively little danger. However, if remote users can execute arbitrary programs, they can potentially do more damage.

We are now finished with our study of computer networks. However, there is still more to learn. New subjects are developing all the time, for example, network management (Klerer, 1988), and old ones are evolving. The reader who wishes to keep up-to-date on this fascinating subject will certainly have plenty to do. See Chap. 10 for some suggested readings to get started.

9.7. SUMMARY

In this chapter we have examined some of the key aspects of the application layer. We started with ACSE and CCR, which manage associations and two-phase commit, respectively. Many applications use these service elements to do their jobs.

Next we looked at file servers. All file servers have an implicit model of what a file is and what operations can be performed on it. The OSI virtual filestore regards a file as a tree of independently addressable elements. It also associates a large number of attributes with a file, and provides operations to read and modify them.

If two or more users try to access the same file at the same time, problems, including deadlocks can arise. For this reason, file servers use concurrency control algorithms to manage simultaneous access. To provide more reliable service, some file servers replicate their files. This practice makes reads easier but updates harder. Various algorithms have been devised to handle multicopy updates, including primary copy and voting.

Another major application is electronic mail. Electronic mail systems have two major components, the user agent and the message transfer agent. Mail sent electronically consists of the content (header plus body) and the envelope. The user agent is concerned with the content whereas the message transfer agent deals with the envelope.

Most mail systems have a facility to manage mailboxes for their users. Commands are typically provided to read, answer, move, and delete messages, as well as send new mail.

Virtual terminals are used to allow screen-oriented software to be written in a terminal independent way. Three types of terminals exist, each with their own properties: scroll mode, page mode, and form mode. Scroll mode terminals can be handled with a stream-oriented character protocol. Page mode terminals can be dealt with using a scheme such as termcap, in which programs execute abstract terminal commands that are mapped onto device dependent escape sequences.

Form mode terminals are typically modeled using a two- or three-dimensional character array. Characters can be entered into the array either from the keyboard

or by the computer. The problem of simultaneous access is solved in the asynchronous model by having two arrays and in the synchronous model by controlling access with a token.

Finding the name of a network user or service in a large network requires a directory service. For a small network, having a file mapping user or service names onto network addresses is sufficient. For a large network, a hierarchical directory system in which objects can be located by various attributes is more satisfactory.

Remote job entry is another application layer function. The OSI JTM service provides a way for a user to collect programs and input files from various source agencies and have them shipped to an execution agency for processing. The results can then be stored with sink agencies and reports returned to the person who submitted the job.

Other applications are picture storage and transfer, videotex and teletext. Pictures are built up from basic elements, such as polylines and ellipses. By combing the basic elements, complex pictures can be represented in a small number of bits. Videotex and teletext are telematic services for home and office use. Videotex allows remote access to large public data bases, and teletext allows for broadcasting of mass market information.

PROBLEMS

1. If CCR and two-phase commit are used on computers that may crash, serious problems may develop. Give an example of such a scenario and propose a solution to the problem.
2. A UNIX file is a linear sequence of bytes, numbered from 0 up to some maximum. An operation, *LSEEK*, is provided to move the file pointer to an arbitrary byte position, after which data can be read or written. Discuss how an OSI virtual filestore could be implemented on a UNIX system to provide remote access to the UNIX files using the OSI primitives.
3. Earlier versions of FTAM only allowed a single FADU to be read or written in one operation. How would such a restriction affect the solution to the previous problem?
4. Some of the OSI virtual filestore attributes can be changed by the owner of a file and others cannot be. For example, the owner can change the *Account number* attribute, but not the *Owner* attribute. Give a plausible reason for this restriction.
5. A replicated file has 5 servers. List all the possible ways of choosing the read quorum and the write quorum for the voting algorithm.
6. Consider the previous problem again. Suppose that reads are very common and writes are very infrequent. Which choice is most efficient?

7. Devise a method by which users of a stateless file server can gain exclusive access to files for update purposes without using locks.

8. The FTAM *F-SELECT* and *F-OPEN* primitives both allow concurrency control to be specified. However, if *F-SELECT* has already asked for and been granted exclusive access, it is not permitted for *F-OPEN* to change to shared access upon entering the open regime, even though this cannot fail. Why was such a rule introduced?

9. In Fig. 9-11 the message storage area (mailbox) is part of the message transfer agent rather than the user agent. Inasmuch as manipulating the mailbox is something that users do from terminals, it would seem to be in the wrong place. Why was it placed in the message transfer agent?

10. Electronic mail systems use people's names in some of the header fields. Discuss the possibility of breaking up names into standard components with an eye to finding a standard representation. Consider all the components that are in common use and would cause problems if they could not be handled.

11. The originator is mentioned in both the envelope and header in the electronic mail example given in the text. Why?

12. In Fig. 9-19 we listed 18 commands for manipulating the character array of a virtual terminal. Leaving the question of efficiency aside, devise the smallest set of primitives that can perform all the functions of the commands in the figure.

13. The OSI virtual terminal committee has been discussing adding a "ripple delete" feature to the virtual terminal service. This feature would allow a block of text to be deleted, and have the text following it move leftward and upward to take the place of the deleted text. What could such a feature be used for?

14. Explain in your own words what net-effecting is good for.

15. Look at the list of termcap escape sequences (not the Booleans or integers) given in Fig. 9-17. Are there any things termcap can do that cannot be handled directly or simulated easily with the OSI VTS?

16. The OSI directory service requires each directory to have a key attribute. Why? (Hint: think about implementing telephone directories with millions of entries.)

17. Which attributes are inherited by DEC's marketing department in Fig. 9-21?

18. How many picture drawing commands of the type discussed in this chapter are needed to draw a regular hexagon?

19. Imagine that 5 percent of a 6 MHz television channel is available for teletext, and that teletext pages are 20 lines of 40 characters, with the characters set in ASCII using binary encoding. How many pages can the system handle and still have an average response time under 1 second?

20. Consider an alphageometric videotex system operating on a 256×256 bit map display. Design a byte-oriented protocol for transmitting polylines and circles.

21. Can you see any advantage to adding the type rectangle to the protocol of the previous problem? After all, a rectangle can be drawn as a polyline.

22. Write a program that simulates that subset of the OSI virtual filestore described in this chapter. The simulator should take the form of a program running on your computer that accepts filestore commands and carries them out. Invent a simple protocol for requests and replies.

23. Write a simple user agent that expects to find mail in a certain directory. Each file is one message. The user agent should provide a way to get an overview of the currently available mail. It should also allow users to read and delete specific messages. Furthermore, it should allow users to compose messages, by calling an editor. When the editor exits, the user agent should get control again and send the mail by putting it in a system spooling directory.

24. Write a program that provides that subset of the OSI directory service described in this chapter. There should be commands to make entries, delete entries and look up entries, plus whatever other commands you need (e.g., managing directories). To make it simpler, an entry should only contain a name, in the form of a 32-character ASCII string, and network address, in the form of a 16-digit decimal number.